The National University of Computer and Emerging Sciences

Architectures of Convolutional Neural Networks

Deep Learning

Goals

- Todays Lecture
  - The Inception Family
    - GoogLeNet (V1), V2, V3, V4, Inception-ResNet
  - Resent
  - Xception

Motivation: Why GoogLeNet?

- Before GoogLeNet, deep CNNs like **AlexNet** and **VGGNet** showed that increasing **depth** improved **performance**. However, going deeper also meant:
  - Huge parameter counts (VGGNet had ~138 million).
  - Overfitting on small datasets.
  - Inefficient computation.
  - Vanishing gradients during training.
- So the question was: **Can we go deeper without exponentially increasing parameters or compute? Yes**

👉 Enter the **Inception module**, the brilliant core idea of inceptionNet also called GoogLeNet.

The Inception Family

- The Inception network was an important milestone in the development of CNN classifiers
- Prior to its inception, most popular CNNs just stacked convolution layers deeper and deeper, hoping to get better performance.
- The Inception Family is all about going ***wider***.
- In particular, the authors of Inception were interested in the computational efficiency of training larger nets.
- In other words: *how can we scale up neural nets without increasing computational cost?*

The Inception Family

- The popular versions are as follows:
  - Inception v1.
  - Inception v2
  - Inception v3.
  - Inception v4 and Inception-ResNet.
- Each version is an iterative improvement over the previous one.
- A new building block for deep nets was introduced, now known as the "*Inception module.*"

Inception v1 (GoogLeNet)

Going deeper with **convolutions**

C Szegedy, W Liu, Y Jia, P Sermanet... - Proceedings of the ..., 2015 - cv-foundation.org

We propose a **deep** convolutional neural network architecture codenamed Inception that achieves the new state of the art for classification and detection in the ImageNet Large-Scale ...

☆ Save 99 Cite Cited by 65241 Related articles All 53 versions »

Inception v1 (GoogLeNet)

Going Deeper with Convolutions

Christian Szegedy[1], Wei Liu[2], Yangqing Jia[1], Pierre Sermanet[1], Scott Reed[3],
Dragomir Anguelov[1], Dumitru Erhan[1], Vincent Vanhoucke[1], Andrew Rabinovich[4]
[1]Google Inc. [2]University of North Carolina, Chapel Hill
[3]University of Michigan, Ann Arbor [4]Magic Leap Inc.
[1]{szegedy,jiayq,sermanet,dragomir,dumitru,vanhoucke}@google.com
[2]wliu@cs.unc.edu, [3]reedscott@umich.edu, [4]arabinovich@magicleap.com

Abstract

We propose a deep convolutional neural network architecture codenamed Inception that achieves the new state of the art for classification and detection in the ImageNet Large-Scale Visual Recognition Challenge 2014 (ILSVRC14). The main hallmark of this architecture is the improved utilization of the computing resources inside the network. By a carefully crafted design, we increased the depth and width of the network while keeping the computational budget constant. To optimize quality, the architectural decisions were based on the Hebbian principle and the intuition of multi-scale processing. One particular incarnation used in our submission for ILSVRC14 is called GoogLeNet, a 22 layers deep network, the quality of which is assessed in the context of classification and detection.

ger and bigger deep networks, but from the synergy of deep architectures and classical computer vision, like the R-CNN algorithm by Girshick et al [6].

Another notable factor is that with the ongoing traction of mobile and embedded computing, the efficiency of our algorithms – especially their power and memory use – gains importance. It is noteworthy that the considerations leading to the design of the deep architecture presented in this paper included this factor rather than having a sheer fixation on accuracy numbers. For most of the experiments, the models were designed to keep a computational budget of 1.5 billion multiply-adds at inference time, so that the they do not end up to be a purely academic curiosity, but could be put to real world use, even on large datasets, at a reasonable cost.

In this paper, we will focus on an efficient deep neural network architecture for computer vision, codenamed Inception, which derives its name from the Network in net-

Inception v1 (GoogLeNet)

- **From where the name Inception comes?**

> *"In this paper, we will focus on an efficient deep neural network architecture for computer vision, code named Inception, which derives its name from (...) the famous "we need to go deeper" internet meme."*

Inception v1 (GoogLeNet)

- **Salient parts** in the image can have extremely **large variation** in size.
- For instance, the area occupied by the dog is different in each image.

Inception v1 (GoogLeNet)

- Because of this huge variation in the location of the information, choosing the ***right kernel size*** for the convolution operation becomes tough.

- A ***larger kernel*** is preferred for information that is distributed more ***globally***, and a ***smaller kernel*** is preferred for information that is distributed more ***locally***.

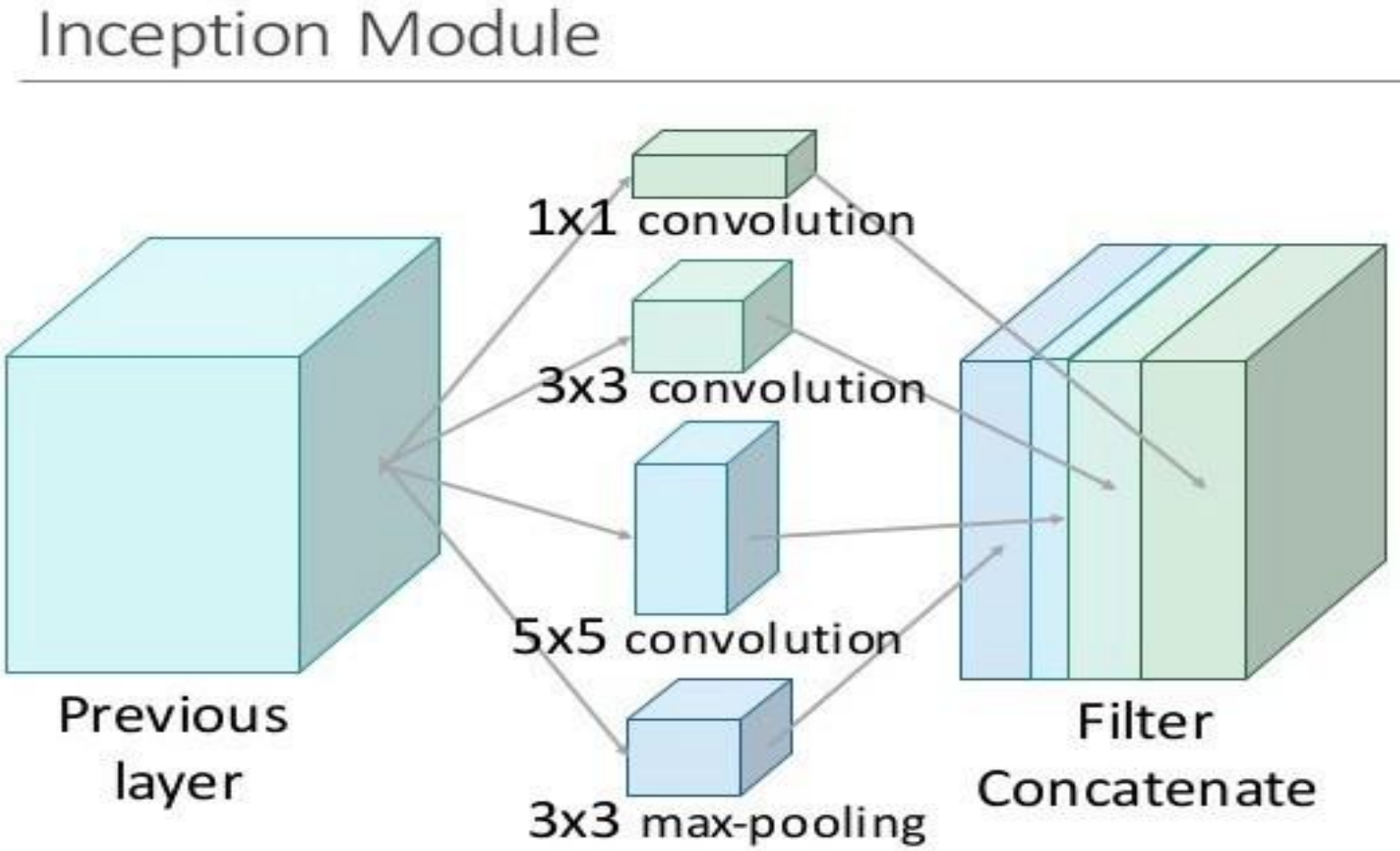
Inception v1 (GoogLeNet)

- In a traditional conv net, each layer extracts information from the previous layer in order to transform the input data into a more useful representation.
- However, each layer type extracts a different kind of information.
- The output of a 5x5 convolutional kernel tells us something different from the output of a 3x3 convolutional kernel, which tells us something different from the output of a max-pooling kernel, and so on and so on.
- At any given layer, how do we know what transformation provides the most "useful" information?
- *Why not let the model chose?*

Inception v1 (GoogLeNet)

- Why not have filters with ***multiple sizes*** operate on the ***same level***? The network essentially would get a bit "***wider***" rather than "deeper"
- An Inception module computes *multiple different transformations* over the same input map in parallel, concatenating their results into a single output.
- In other words, for each layer, Inception does a 5x5 convolutional transformation, *and* a 3x3, *and* a max-pool.
- And the next layer of the model gets to decide if (and how) to use each piece of information.

The Inception Module



Inception v1 (GoogLeNet)

- The increased information density of this model architecture comes with one glaring problem: Drastically increased *computational costs*.
- Not only are large (e.g. 5x5) convolutional filters inherently expensive to compute, stacking multiple different filters side by side greatly increases the number of feature maps per layer.
- For each additional filter added, we have to convolve over *all* the input maps to calculate a single output.

Inception v1 (GoogLeNet)

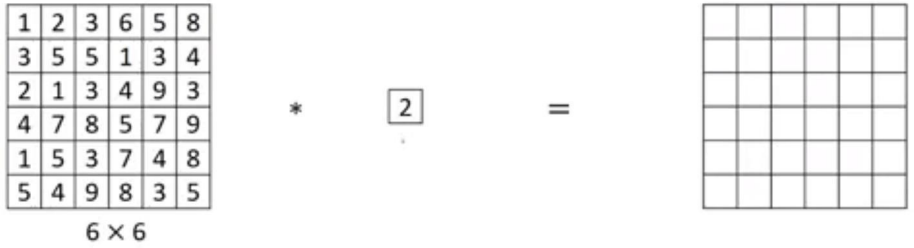
- As stated before, deep neural networks are **computationally expensive**.
- To make it cheaper, the authors **limit** the number of **input channels** by adding an **extra 1x1 convolution** before the 3x3 and 5x5 convolutions.
- Though adding an extra operation may seem counterintuitive, 1x1 convolutions are far more cheaper than 5x5 convolutions, and the reduced number of input channels also help.

Inception v1 (GoogLeNet)

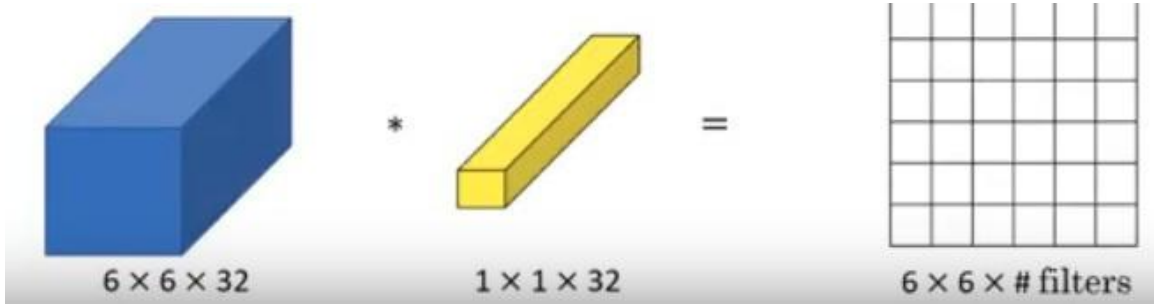
- A 1x1 convolution only looks at one value at a time, but across multiple channels, it can extract spatial information and compress it down to a lower dimension.
- For example, using 20 1x1 filters, an input of size 64x64x100 (with 100 feature maps) can be compressed down to 64x64x20.
- By reducing the number of input maps, the authors of Inception were able to stack different layer transformations in parallel, resulting in nets that were simultaneously deep (many layers) and "wide" (many parallel operations).

Inception v1 (GoogLeNet)

- What does a 1x1 Convolution do?

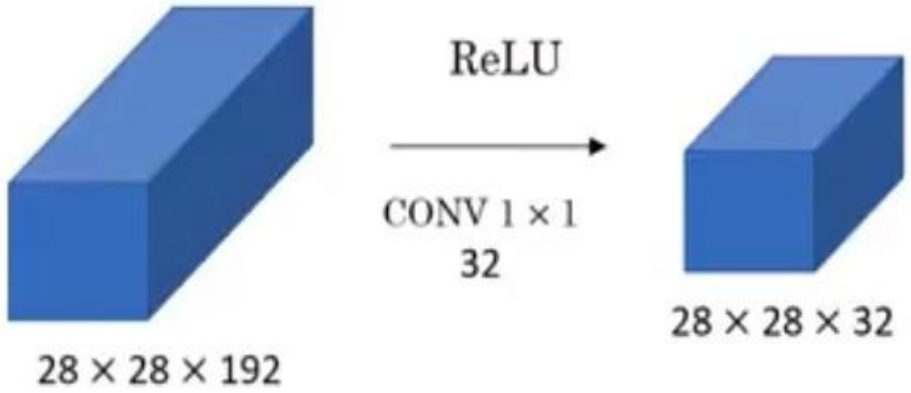


- For single channel – Does not seem to be useful: Similar to multiplication by a scalar
- For multiple channels:



Inception v1 (GoogLeNet)

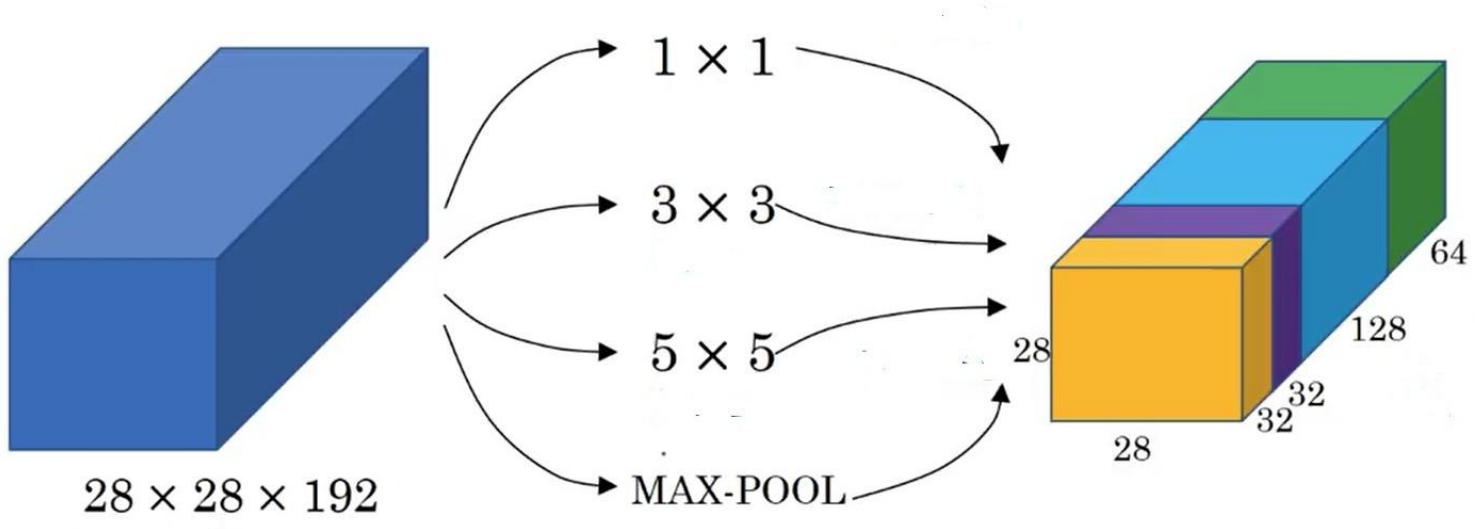
- To reduce spatial dimension – Use Pooling
- To reduce depth – Use 1x1 Convolution



Lin, Min, Qiang Chen, and Shuicheng Yan. "*Network in network.*" *arXiv preprint arXiv:1312.4400* (2013).

Inception v1 (GoogLeNet)

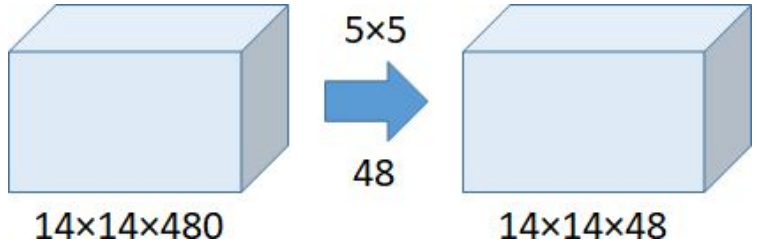
- Let the network decide which filter size is good and whether we need conv or pooling



Inception v1 (GoogLeNet)

- In GoogLeNet, 1×1 convolution is used as a dimension reduction module to reduce the computation.

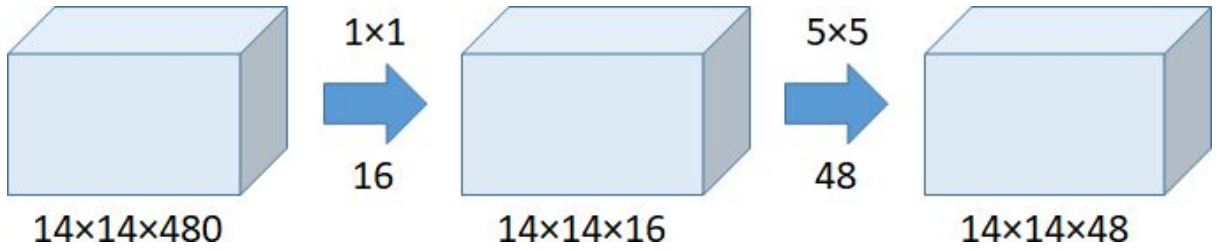
- Example:



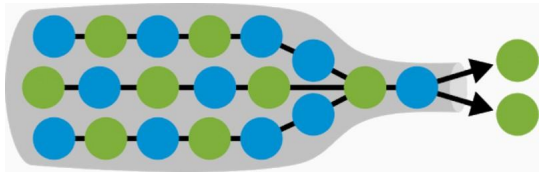
Number of operations = (14×14×48)×(5×5×480) = 112.9M

Inception v1 (GoogLeNet)

- Example:
- **Bottleneck Layer:** Smallest part of the NW



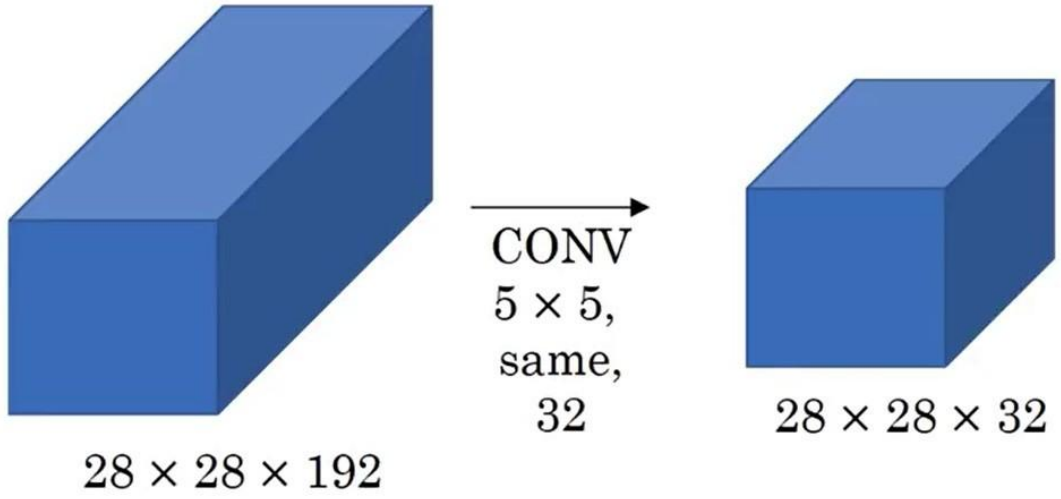
Number of operations for 1×1 = (14×14×16)×(1×1×48) = 1.5M
Number of operations for 5×5 = (14×14×48)×(5×5×16) = 3.8M
Total number of operations = 1.5M + 3.8M = 5.3M



Inception v1 (GoogLeNet)

Example 2

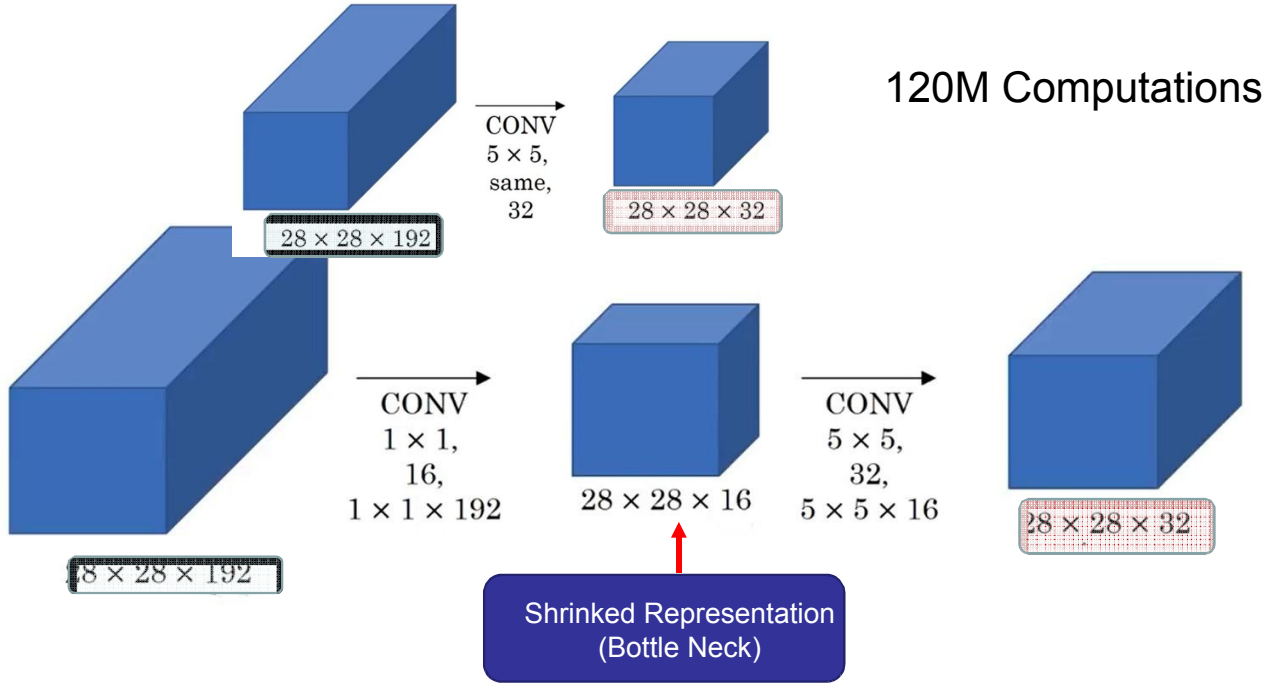
- Computational Cost



- There are 32 filters and each filter is 5x5x192
- We compute 28x28x32 numbers – Computation of each requires 5x5x192 multiplications
- Total multiplications: (28x28x32) x (5x5x192) = 120 M

Inception v1 (GoogLeNet)

Example 2

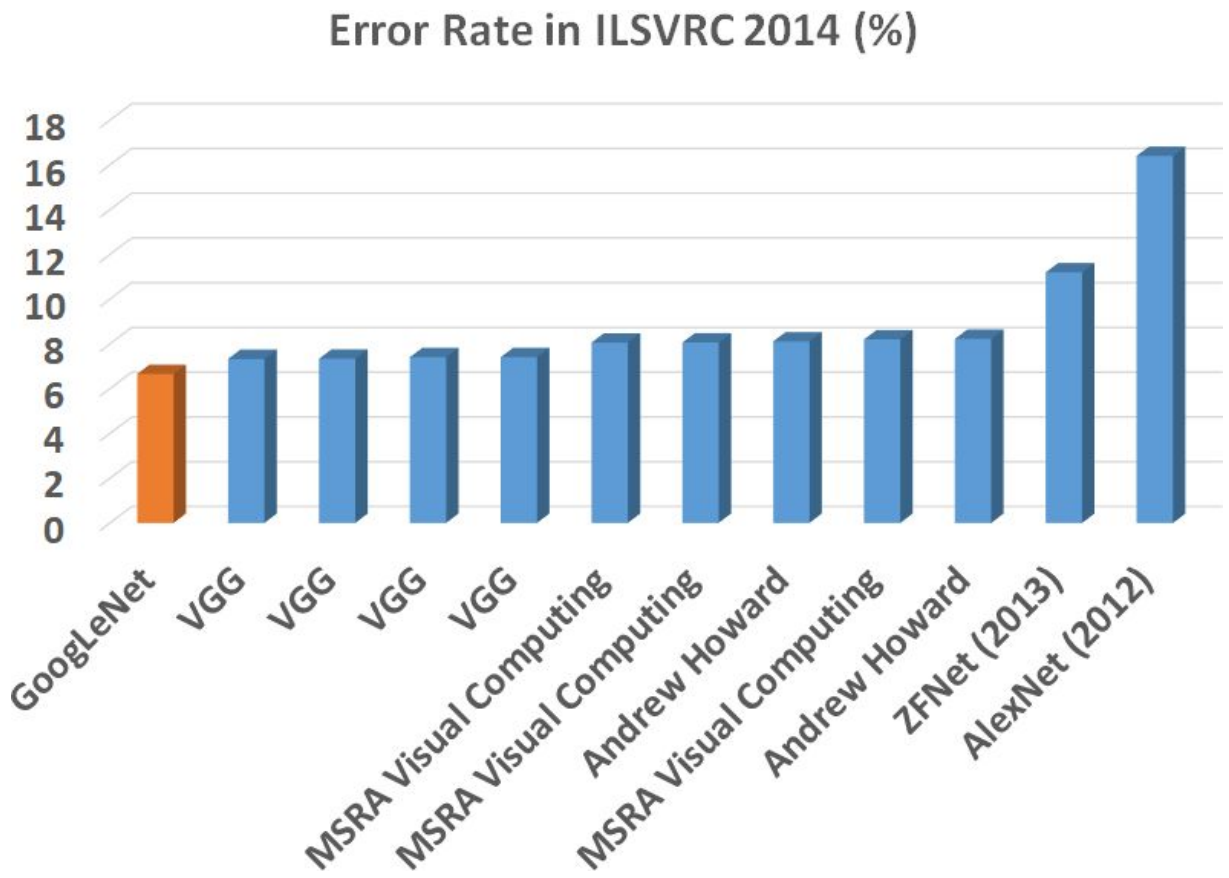


Computations: (28x28x16) x (1x1x192)+ (28x28x32) x (5x5x16) = 12.4M

Inception v1 (GoogLeNet)

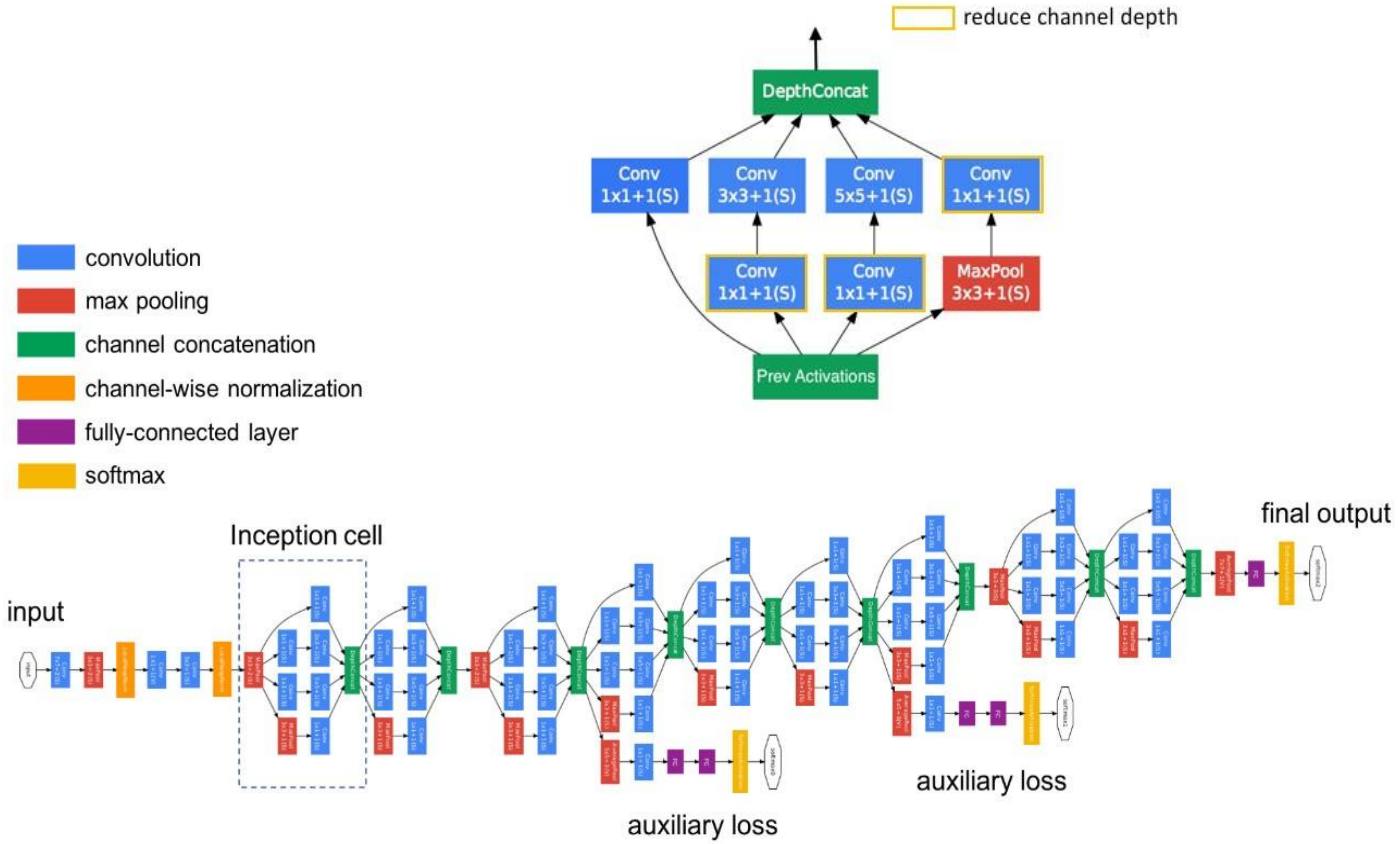
- Using the dimension reduced inception module, a neural network architecture was built (2014) which got the *first place in the 2014* ImageNet competition for classification and detection challenges.
- GoogLeNet has 9 such inception modules stacked linearly.
- GAP (converting 7×7 maps to 1×1) layer after last inception module
- It is *22 layers* deep (27, including the pooling layers).
- It achieved a top-5 error rate with of 6.67%
- It reduces the number of parameters from 60 million (AlexNet) to 4 million.

Inception v1 (GoogLeNet)

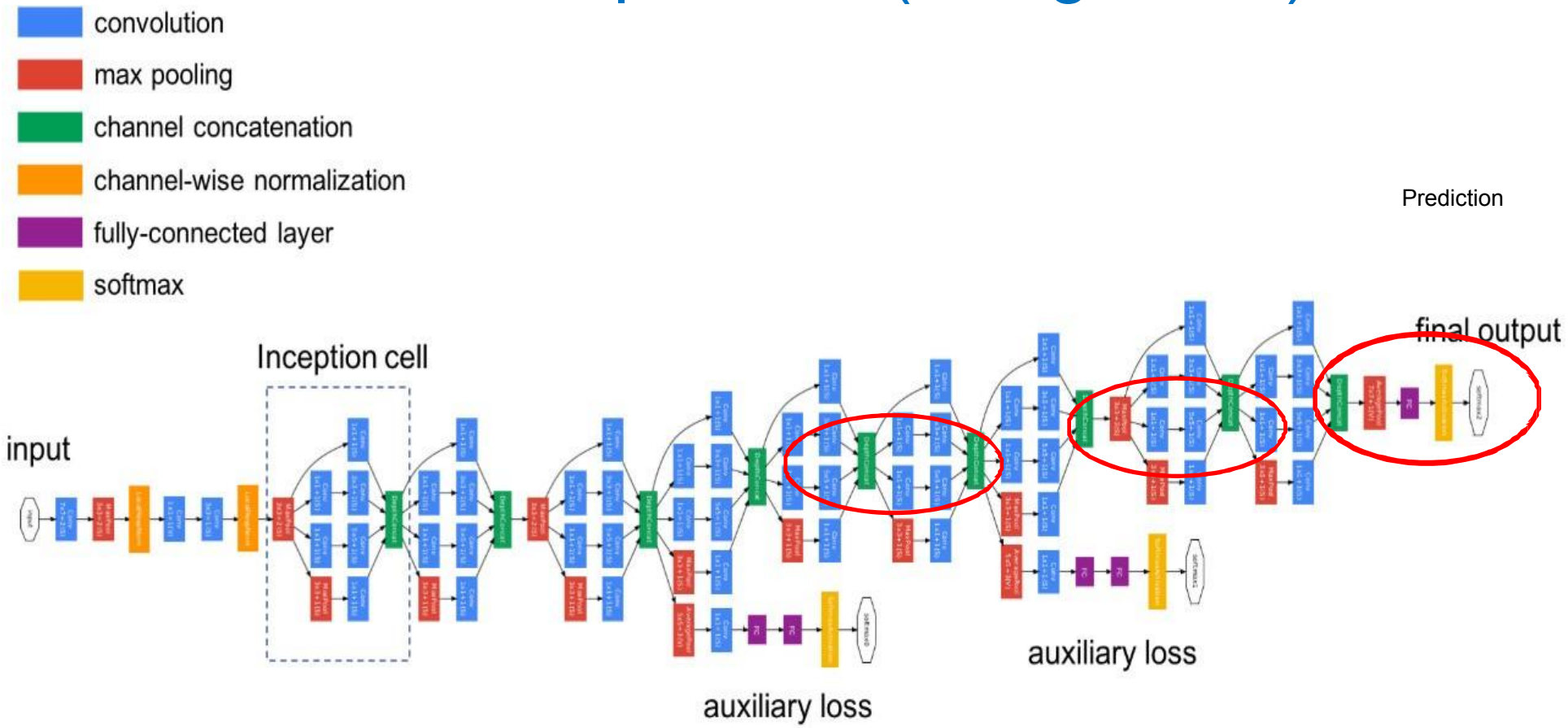


Inception v1 (GoogLeNet)

- Put Inception blocks together to get the network



Inception v1 (GoogLeNet)



Take a hidden layer and use it for prediction (FC+Softmax)
Ensures: Features computed at intermediate layers are not too bad in prediction
A regularization effect

Inception v1 (GoogLeNet)

- The auxiliary classifiers consist of:
  - 5×5 Average Pooling (Stride 3)
  - 1×1 Conv (128 filters)
  - 1024 FC
  - 1000 FC
  - Softmax
- The loss is added to the total loss, with weight 0.3.

```
# The total loss used by the inception net during training.
total_loss = real_loss + 0.3 * aux_loss_1 + 0.3 * aux_loss_2
```

Deeper n/w Problems

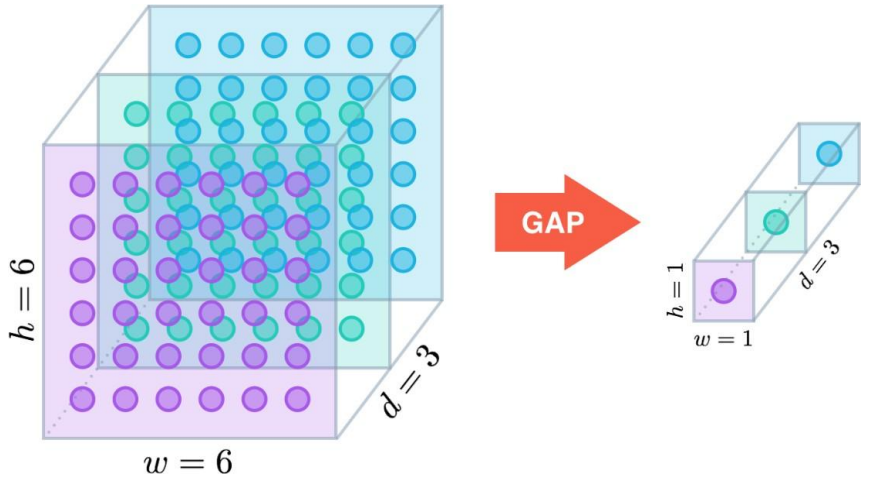
- When we design a deep neural network, we decide the number of layers it will contain and the number of neurons per layer so if the number of layers are more then it might result in following problems :
  - The bigger our model is (more number of layers), the more it will be prone to the problem of overfitting (When number of input feature is large during training)
  - If the number of layers are large, the number of parameters will also increase and hence we'll also be required to increase the computational resources only then it will be able to perform the computation on these parameters.
- So, rather then increasing the computational resource, we can use an Inception network which will minimize the computation costs while also increasing the depth and width of the network.

Key Features of GoogLeNet (Inception V1)

- **1×1 Convolutions**
  - For dimensionality reduction
- **Global Average Pooling**
  - Reduces the model's parameter count and solves overfitting
- **Inception Module**
  - Enables the network to capture features at **multiple scales** effectively
  - Improves representational power without dramatically increasing computation.
- **Auxiliary Classifiers**
  - To address the vanishing gradient problem during training
  - Regularization

Inception v1 (GoogLeNet)

- Working of Global Average Pooling
  - Similar to max pooling layers, GAP layers are used to reduce the spatial dimensions of a three-dimensional tensor.
  - However, GAP layers perform a more extreme type of dimensionality reduction, where a tensor with dimensions *h×w×d* is reduced in size to have dimensions *1×1×d*.
  - GAP layers reduce each h×w feature map to a single number by simply taking the average of all *hw* values.



Structural & Computational Issues with Inception V1

- **Inefficient 5x5 Convolutions**
  - Input dimensions to decrease by a large margin
  - Accuracy decreases
  - Complexity decrease
- **Use factorization**
  - *3x3* convolution to an asymmetric convolution of *1x3* then followed by a *3x1 convolution*

Inception v2

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Sergey Ioffe SIOFFE@GOOGLE.COM
Christian Szegedy SZEGEDY@GOOGLE.COM
Google, 1600 Amphitheatre Pkwy, Mountain View, CA 94043

Abstract

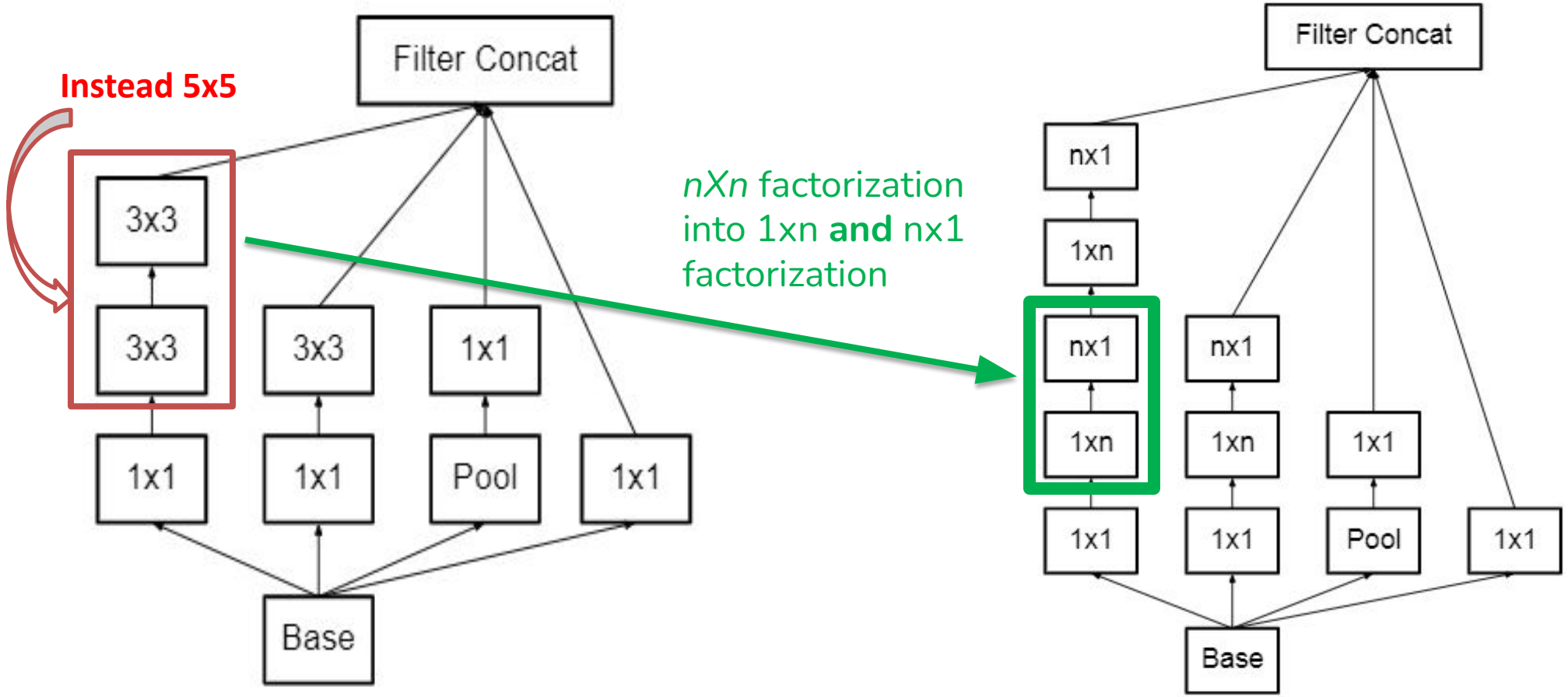
Training Deep Neural Networks is complicated by the fact that the distribution of each layer's inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models with saturating nonlinearities. We refer to this phenomenon as *internal covariate shift*, and address the problem by normalizing layer inputs. Our method draws its strength from making normalization a part of the model architecture and performing the normalization *for each training mini-batch*. Batch Normalization allows us to

minimize the loss

$$\Theta = \arg\min_{\Theta} \frac{1}{N} \sum_{i=1}^{N} \ell(x_i, \Theta)$$

where $x_{1...N}$ is the training data set. With SGD, the training proceeds in steps, at each step considering a *mini-batch* $x_{1...m}$ of size m. Using mini-batches of examples, as opposed to one example at a time, is helpful in several ways. First, the gradient of the loss over a mini-batch $\frac{1}{m} \sum_{i=1}^{m} \frac{\partial \ell(x_i, \Theta)}{\partial \Theta}$ is an estimate of the gradient over the training set, whose quality improves as the batch size increases. Second, computation over a mini-batch can be more efficient than m computations for individual examples on modern computing platforms.

Architectural Changes in Inception V2



Inception v3

Rethinking the Inception Architecture for Computer Vision

Christian Szegedy
Google Inc.
szegedy@google.com

Vincent Vanhoucke
vanhoucke@google.com

Sergey Ioffe
sioffe@google.com

Jon Shlens
shlens@google.com

Zbigniew Wojna
University College London
zbigniewwojna@gmail.com

Abstract

Convolutional networks are at the core of most state-of-the-art computer vision solutions for a wide variety of tasks. Since 2014 very deep convolutional networks started to become mainstream, yielding substantial gains in various benchmarks. Although increased model size and computational cost tend to translate to immediate quality gains for most tasks (as long as enough labeled data is provided for training), computational efficiency and low parameter

networks. VGGNet [18] and GoogLeNet [20] yielded similarly high performance in the 2014 ILSVRC [16] classification challenge. One interesting observation was that gains in the classification performance tend to transfer to significant quality gains in a wide variety of application domains. This means that architectural improvements in deep convolutional architecture can be utilized for improving performance for most other computer vision tasks that are increasingly reliant on high quality, learned visual features. Also, improvements in the network quality resulted in new appli-

Inception v3: Factorizing Convolutions

- Separable Convolutions: The aim of factorizing Convolutions is to ***reduce the number of parameters*** without decreasing the network efficiency.
- Spatially separable convolution decomposes a convolution into two separate operations.
- Example: A Sobel kernel, which is a 3x3 kernel, can be divided into a 3x1 and 1x3 kernel.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

Inception v3: Factorizing Convolutions

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

3x3 Conv

$$\begin{bmatrix} 2 & 3 & 4 \\ 2 & 3 & 4 \\ 2 & 3 & 4 \end{bmatrix} * \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \mathbf{8}$$

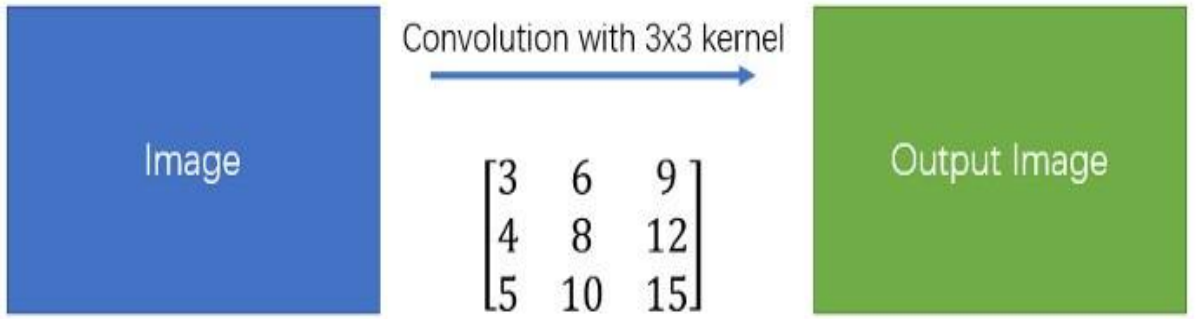
3x1 + 1x3 Conv

$$\begin{bmatrix} 2 & 3 & 4 \\ 2 & 3 & 4 \\ 2 & 3 & 4 \end{bmatrix} * \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 8 & 12 & 16 \end{bmatrix}$$

$$\begin{bmatrix} 8 & 12 & 16 \end{bmatrix} * \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} = \mathbf{8}$$

Inception v3: Factorizing Convolutions

Simple Convolution

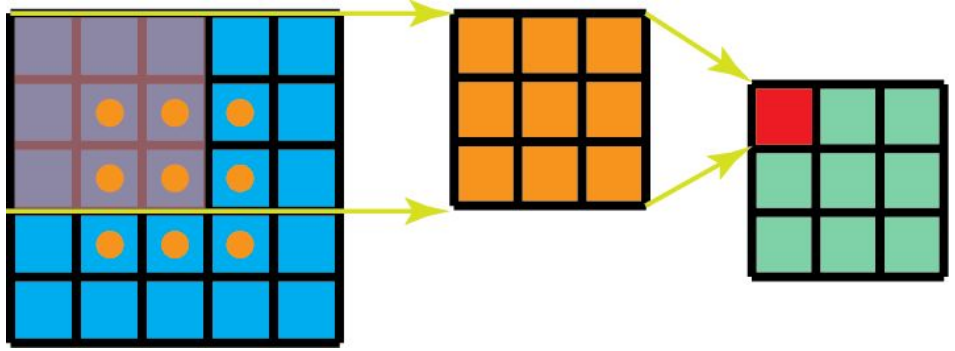


Spatial Separable Convolution

Inception v3: Factorizing Convolutions

- In convolution, the 3x3 kernel directly convolves with the image.
- In spatially separable convolution, the 3x1 kernel first convolves with the image. Then the 1x3 kernel is applied. This would require *6 instead of 9 parameters* while doing the same operations.
- Moreover, we need *less matrix multiplications* in spatially separable convolution than convolution.

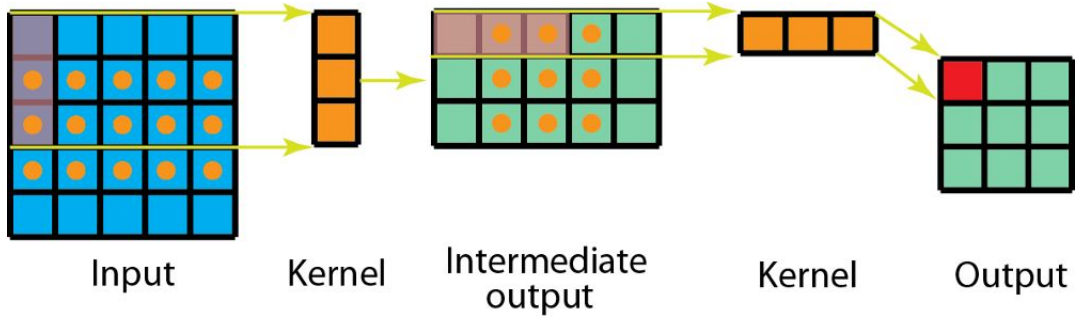
- Example: Convolution on a 5 x 5 image with a 3 x 3 kernel (stride=1, padding=0) requires scanning the kernel at 3 positions horizontally (and 3 positions vertically): i.e. 9 positions in total
- At each position, 9 element-wise multiplications are applied. Overall, that's *9 x 9 = 81* multiplications.



Inception v3: Factorizing Convolutions

- For spatially separable convolution, we first apply a 3 x 1 filter on the 5 x 5 image.
- We scan such kernel at 5 positions horizontally and 3 positions vertically. That's *5 x 3=15* positions in total (indicated as dots on the image below)

- At each position, 3 element-wise multiplications are applied. That is *15 x 3 = 45* multiplications. We now obtained a 3 x 5 matrix.
- This matrix is now convolved with a 1 x 3 kernel, which scans the matrix at 3 positions horizontally and 3 positions vertically. For each of these 9 positions, 3 element-wise multiplications are applied. This step requires *9 x 3=27* multiplications.
- Thus, overall, the spatially separable convolution takes *45 + 27 = 72* multiplications, which is less than convolution.

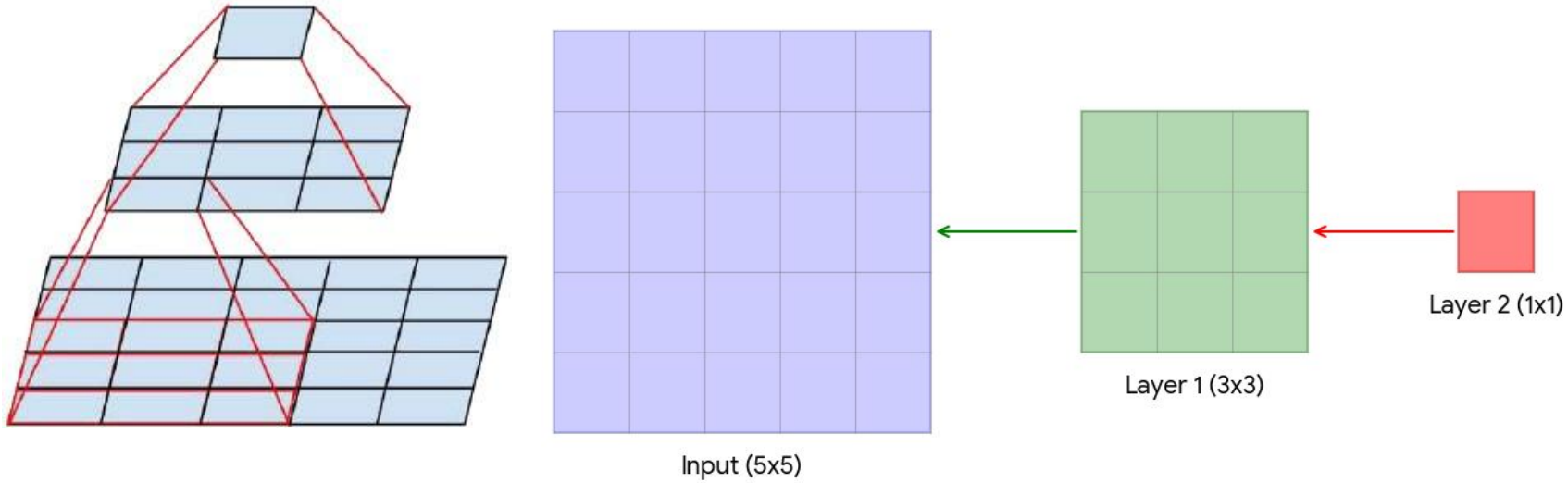


Inception v3: Factorizing Convolutions

- Although spatially separable convolutions save cost, it is not very frequently used in deep learning.
- If we replace all traditional convolutions by the spatially separable convolution, we limit ourselves for searching all possible kernels during training.
- The training results may be sub-optimal.

Inception v3

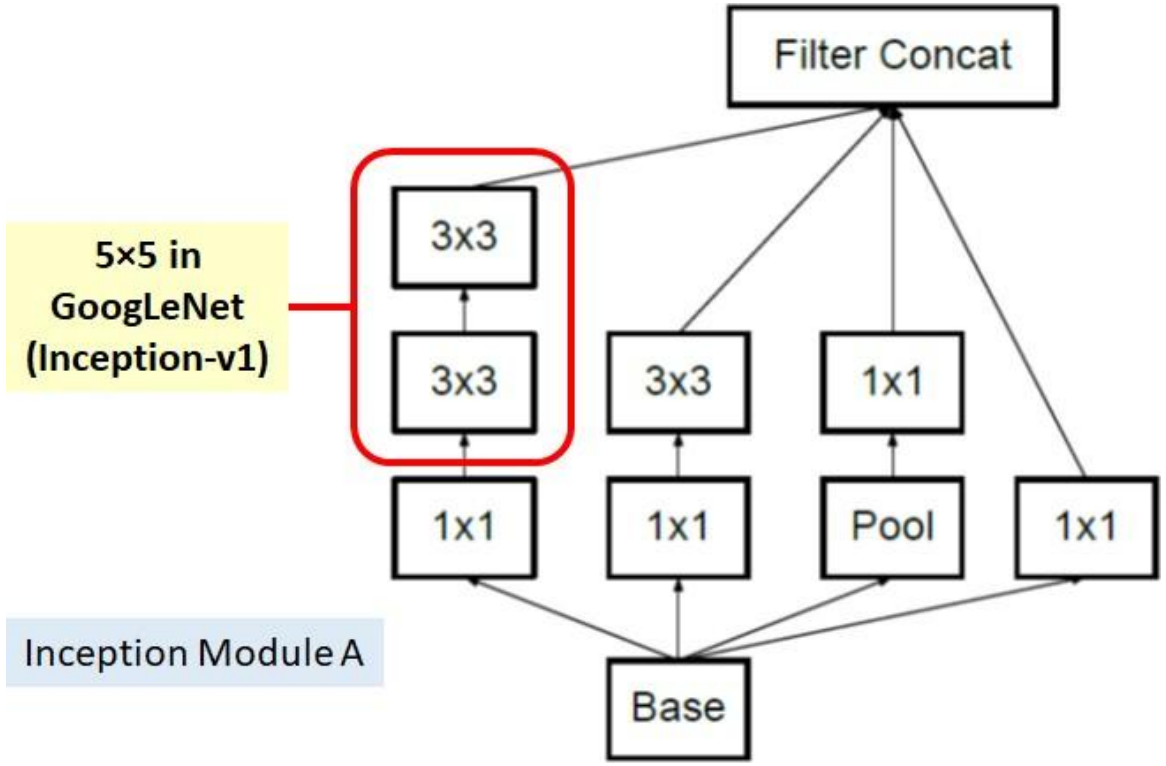
- Factorizing Convolutions:
- – Two 3×3 convolutions replace one 5×5 convolution as follows:



By using **1 layer of 5×5 filter**, number of parameters = **5×5=25**
By using **2 layers of 3×3 filters**, number of parameters = **3×3+3×3=18**
Number of parameters is reduced by 28%

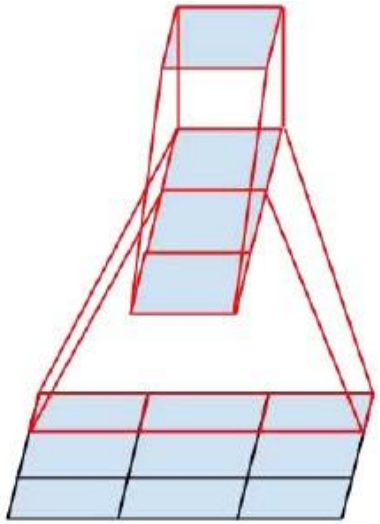
Inception v3

- With this technique, one of the new Inception modules (Let's call it **Inception Module A**) becomes:



Inception v3

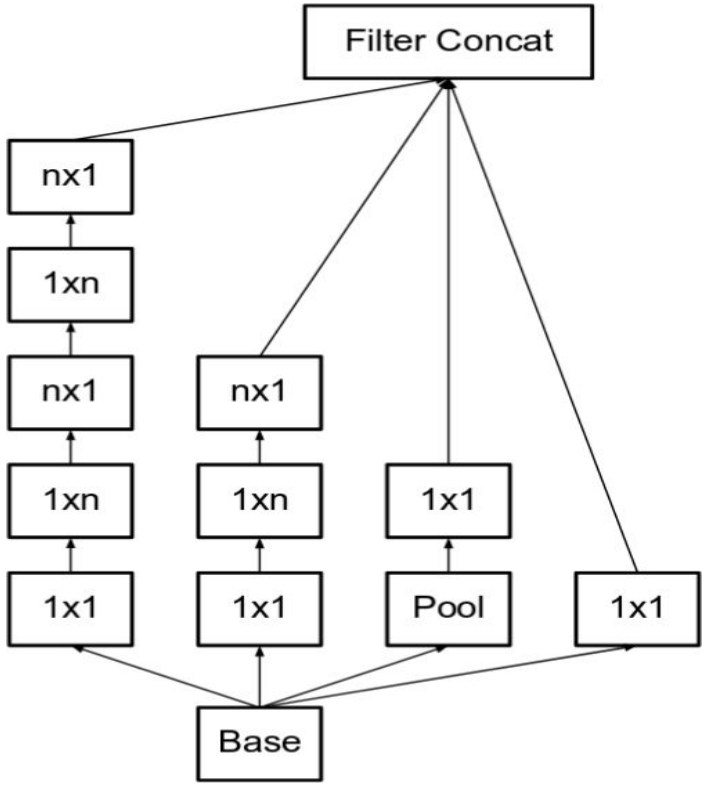
- One 3×1 convolution followed by one 1×3 convolution replaces one 3×3 convolution as follows:



By using **3×3 filter**, number of parameters = **3×3=9**
By using **3×1 and 1×3 filters**, number of parameters = **3×1+1×3=6**
Number of parameters is reduced by 33%

Inception v3

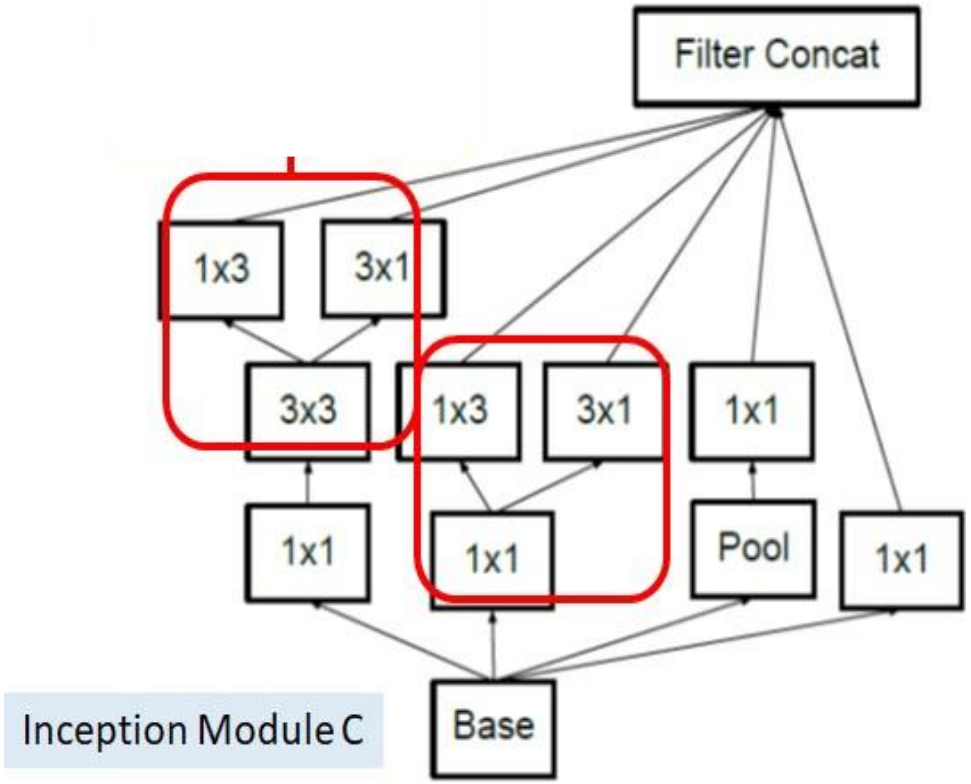
- With this technique, one of the new Inception modules (Let's call it **Inception Module B**) becomes:



Here, put **n=3** to obtain the equivalent of the previous image. The left-most 5x5 convolution can be represented as two 3x3 convolutions, which in turn are represented as 1x3 and 3x1 in series

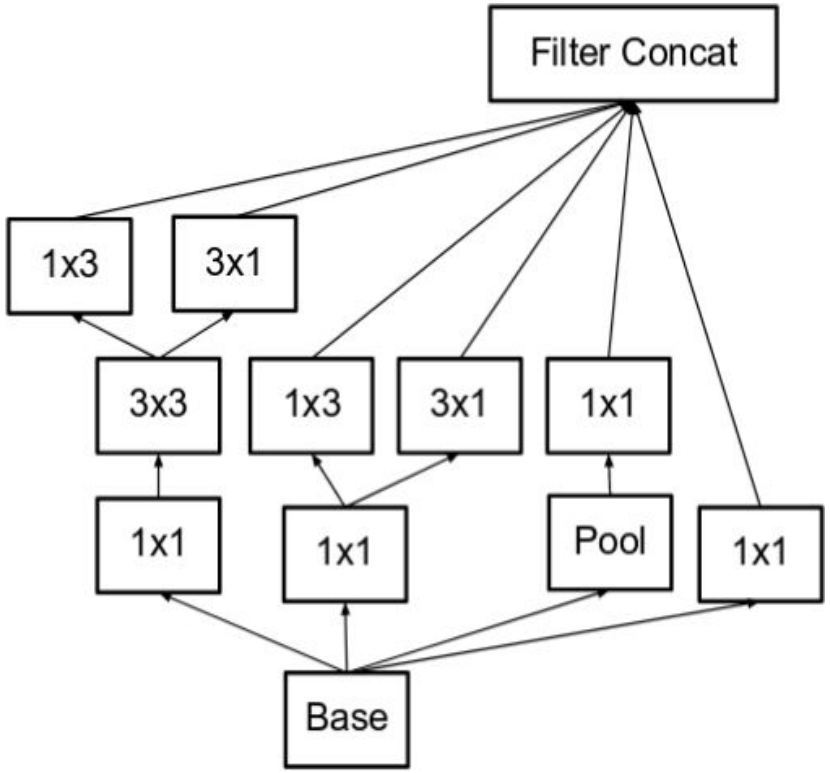
Inception v3

- And **Inception module C** is also proposed as follows:

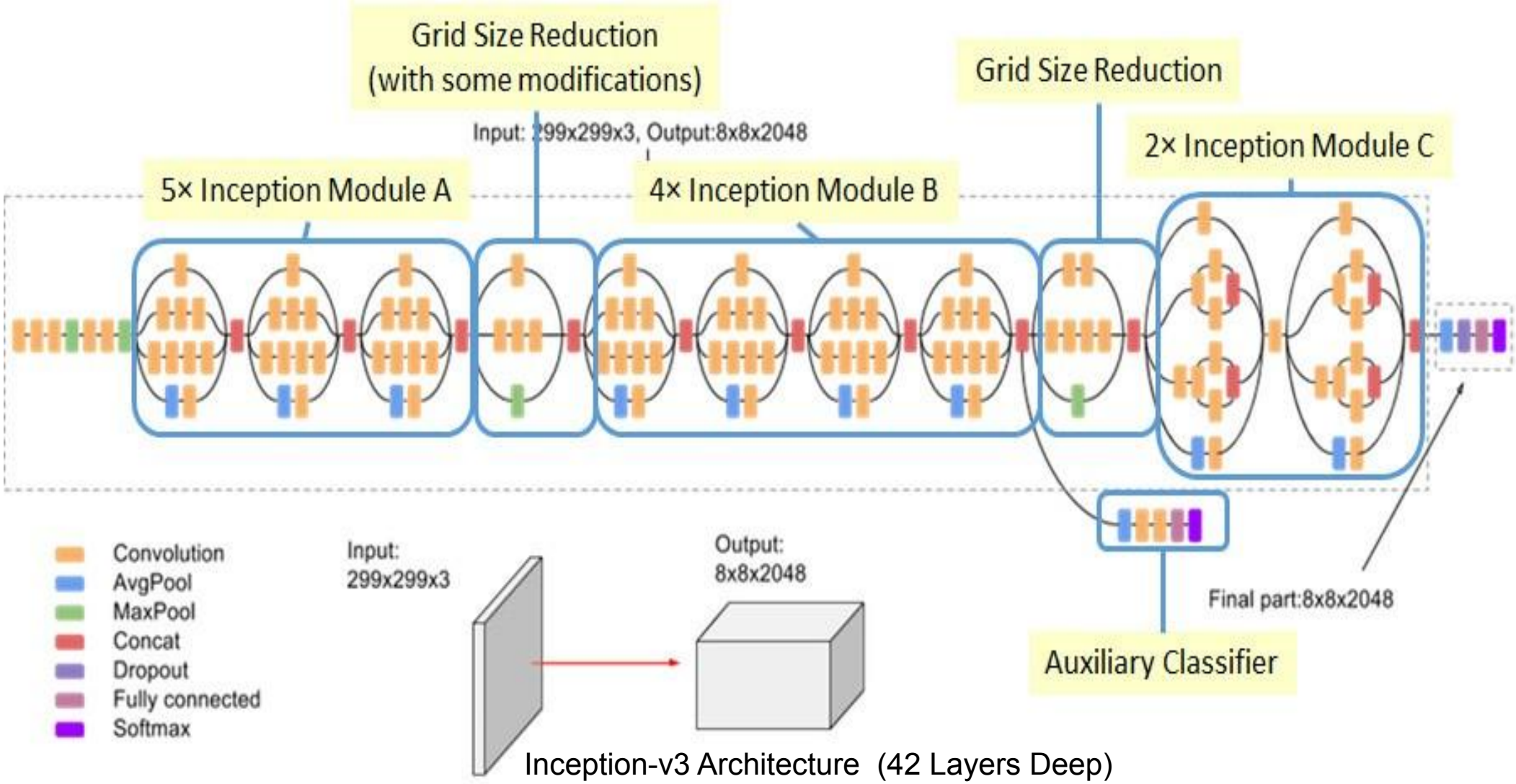


Inception v3

- The **filter banks** in the module were **expanded** (made wider instead of deeper) to remove the representational bottleneck.



Inception v3



Inception-v3 Architecture (42 Layers Deep)

Architectural Changes in Inception V3

- Inception V3 is similar to and contains all the features of Inception V2 with following changes/additions:
  - Use of RMSprop optimizer.
  - Batch Normalization in the fully connected layer of Auxiliary classifier.
  - Use of 7x7 factorized Convolution
  - Label Smoothing Regularization
    - It is a method to regularize the classifier by estimating the effect of label-dropout during training. It prevents the classifier to predict a class too confidently. The addition of label smoothing gives 0.2% improvement from the error rate.

Inception v4 and InceptionResNet

Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning

Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, Alexander A. Alemi
Google Inc.
1600 Amphitheatre Parkway
Mountain View, CA

Abstract

Very deep convolutional networks have been central to the largest advances in image recognition performance in recent years. One example is the Inception architecture that has been shown to achieve very good performance at relatively low computational cost. Recently, the introduction of residual connections in conjunction with a more traditional architecture has yielded state-of-the-art performance in the 2015 ILSVRC challenge; its performance was similar to the latest generation Inception-v3 network. This raises the question: Are there any benefits to combining Inception architectures with residual connections? Here we give clear empirical evidence that training with residual connections accelerates the training of Inception networks significantly. There is also some evidence of residual Inception networks outperforming similarly expensive Inception networks without residual connections by a thin margin. We also present several new streamlined architectures for both residual and non-residual Inception networks. These variations improve the single-frame recognition performance on the ILSVRC 2012 classification task significantly. We further demonstrate how proper activation scaling stabilizes the training of very wide

same neural network architecture "AlexNet" (Krizhevsky, Sutskever, and Hinton 2012) has been applied to a large number of application domains with good results.

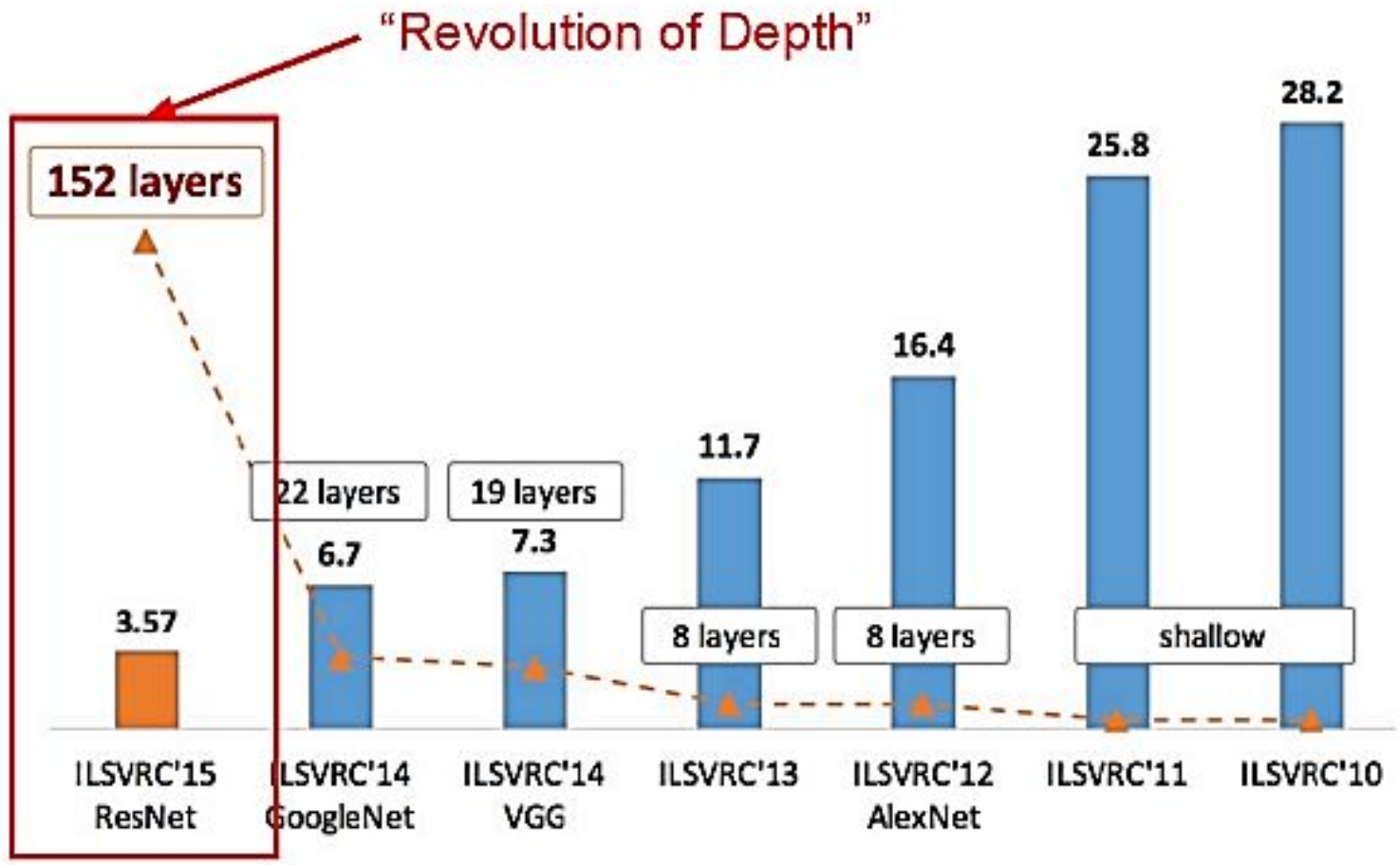
The same architectures that work well for object detection can be applied successfully to a wide variety of computer vision tasks, including object-detection (Girshick et al. 2014), segmentation (Long, Shelhamer, and Darrell 2015), human pose estimation (Toshev and Szegedy 2014), video classification (Karpathy et al. 2014), object tracking (Wang and Yeung 2013), and super-resolution (Dong et al. 2014). These examples are but a few of the multitude of applications to which deep convolutional networks have been very successfully applied ever since. Moreover it has been demonstrated that architectural improvements of convolutional networks that target recognition performance tend to translate to performance gains for the other tasks as well.

This universal applicability motivates our focus on recognition models for the widely used ILSVRC12 object-recognition benchmark (Russakovsky et al. 2014) on which the task is to classify the images into one (or five) of a

GoogLeNet/InceptionNet

- Introduced the idea that CNN layers didn't always have to be stacked up sequentially.
- Coming up with the Inception module, the authors showed that a creative structuring of layers can lead to improved performance and computationally efficiency.

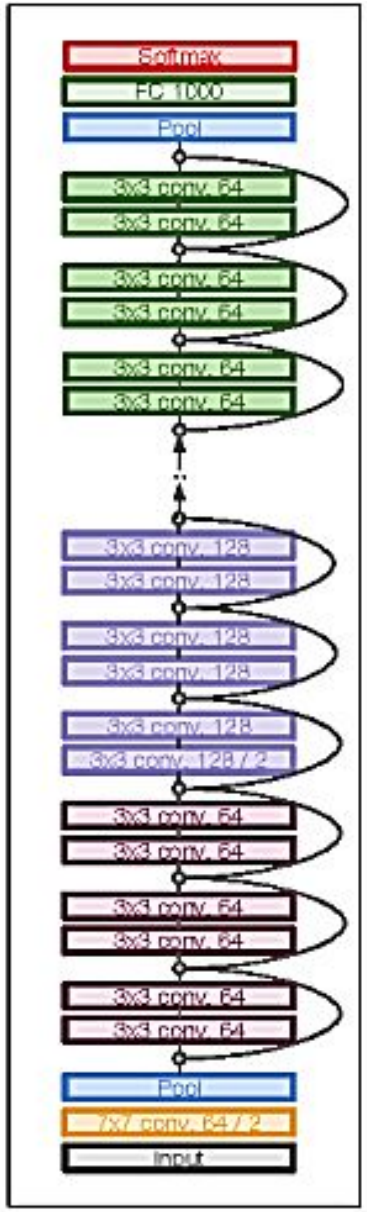
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



ResNet

- Deep Residual Learning for Image Recognition - Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun; 2015
- Extremely deep network – 152 layers
- Deeper neural networks are more difficult to train.
- Deep networks suffer from vanishing and exploding gradients.
- Present a residual learning framework to ease the training of networks that are substantially deeper than those used previously.

[He et al., 2015]



ResNet

- ILSVRC'15 classification winner (3.57% top 5 error, humans generally hover around a 5-10% error rate)
  Swept all classification and detection competitions in ILSVRC'15 and COCO'15!

Slide taken from Fei-Fei & Justin Johnson & Serena Yeung. Lecture 9.

[He et al., 2015]

ResNet

- What happens when we continue stacking deeper layers on a convolutional neural network?

- 56-layer model performs worse on both training and test error

-> The deeper model performs worse (not caused by overfitting)!

Slide taken from Fei-Fei & Justin Johnson & Serena Yeung. Lecture 9.

[He et al., 2015]

ResNet

- **Hypothesis**: The problem is an optimization problem. Very deep networks are harder to optimize.
- **Solution**: Use network layers to fit residual mapping instead of directly trying to fit a desired underlying mapping.

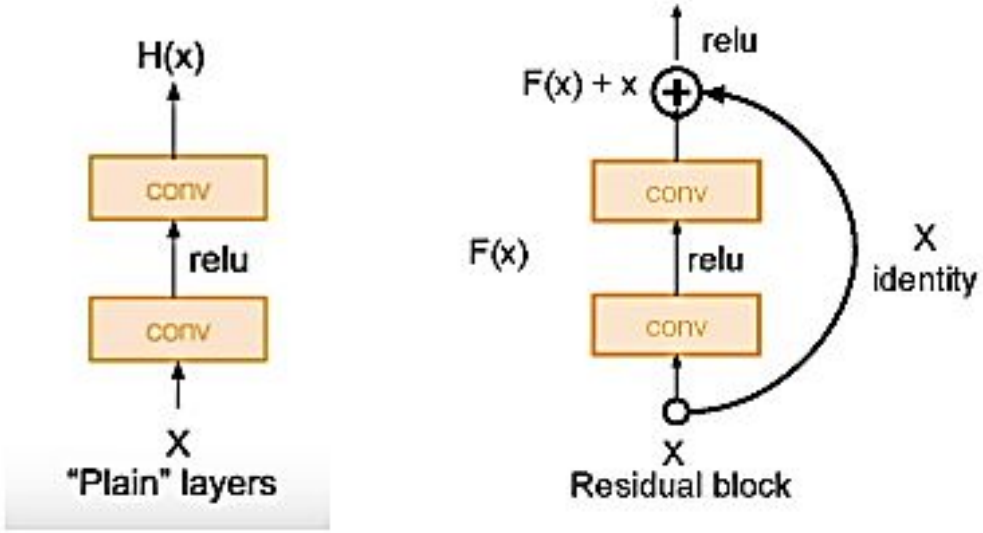
- We will use **skip connections** allowing us to take the activation from one layer and feed it into another layer, much deeper into the network.
- Use layers to fit residual $F(x) = H(x) - x$ instead of $H(x)$ directly

ResNet

Residual Block

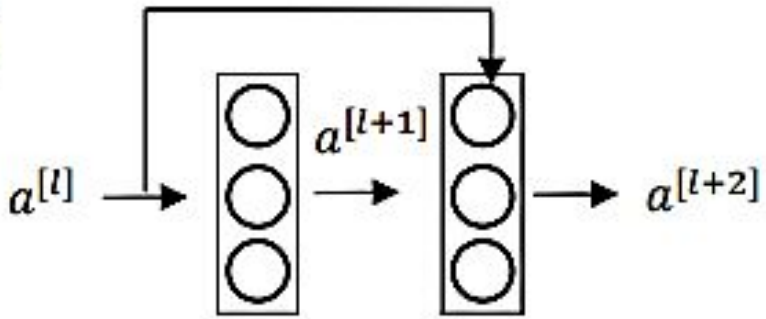
Input x goes through conv-relu-conv series and gives us F(x). That result is then added to the original input x. Let's call that H(x) = F(x) + x.

In traditional CNNs, H(x) would just be equal to F(x). So, instead of just computing that transformation (straight from x to F(x)), we're computing the term that we have to *add*. F(x), to the input, x.



[He et al., 2015]

ResNet



Short cut/ skip connection

$a^{[l]}$ → **Linear** → **ReLU** → **Linear** → **ReLU** → $a^{[l+2]}$

$a^{[l+1]}$

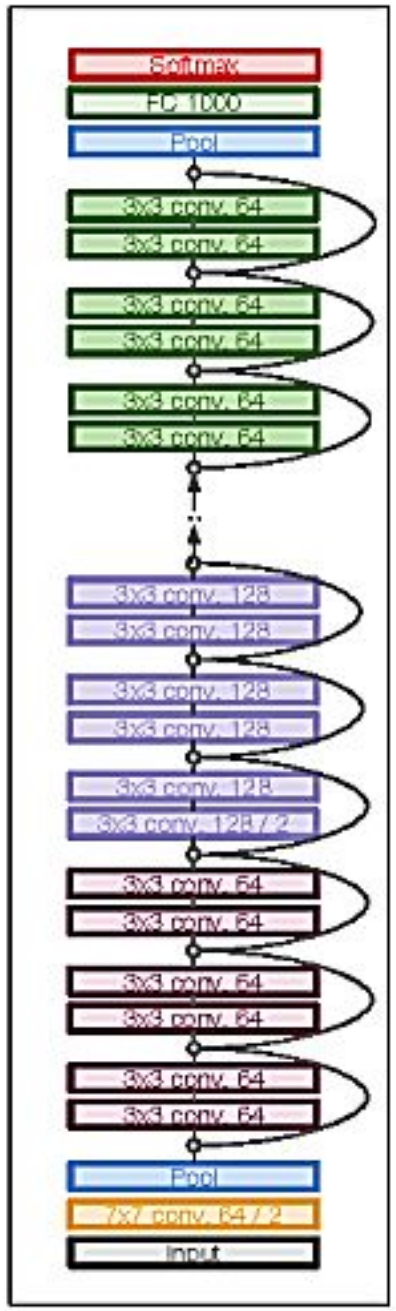
$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

$$a^{[l+1]} = g(z^{[l+1]})$$

$$a^{[l+2]} = g(z^{[l+2]})$$

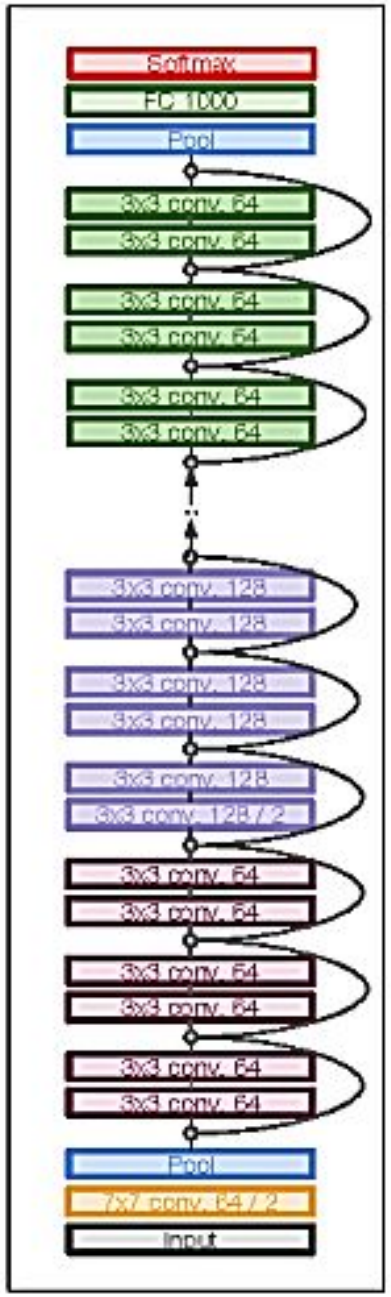
$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]}) = g(W^{[l+2]} a^{[l+1]} + b^{[l+2]} + a^{[l]})$$



ResNet

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (in each dimension)
- Additional conv layer at the beginning
- No FC layers at the end (only FC 1000 to output classes)



ResNet

- Total depths of 34, 50, 101, or 152 layers for ImageNet
- For deeper networks (ResNet-50+), use "bottleneck" layer to improve efficiency (similar to GoogLeNet)

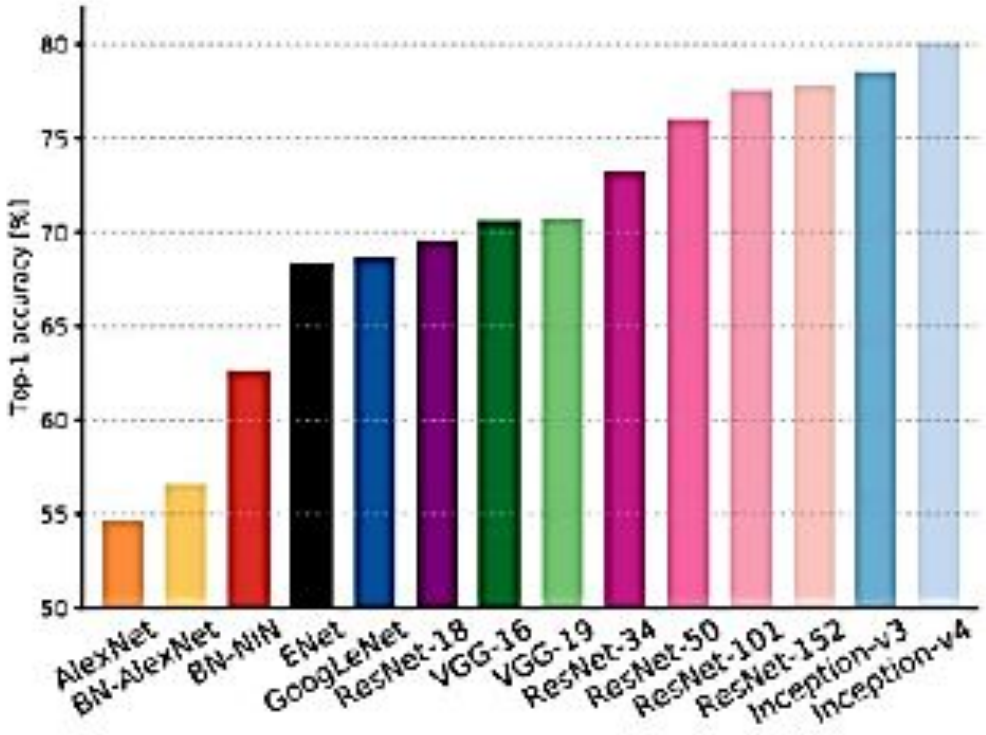
ResNet

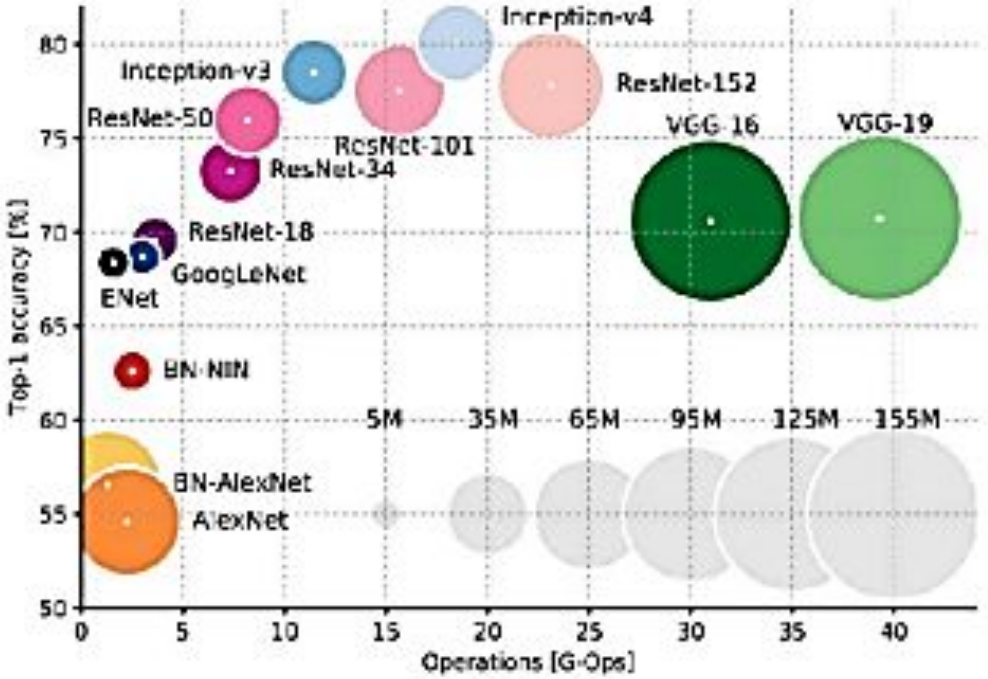
Experimental Results:

- Able to train very deep networks without degrading
- Deeper networks now achieve lower training errors as expected

- The **best** CNN architecture that we currently have and is a great innovation for the idea of residual learning. Even better than human performance!

Accuracy comparison





Xception Network

[PDF] Xception: Deep learning with depthwise separable convolutions

F Chollet - arXiv preprint, 2017 - openaccess.thecvf.com

We present an interpretation of Inception modules in convolutional neural networks as being an intermediate step in-between regular convolution and the depthwise separable convolution operation (a depthwise convolution followed by a pointwise convolution). In this light, a depthwise separable convolution can be understood as an Inception module with a maximally large number of towers. This observation leads us to propose a novel deep convolutional neural network architecture inspired by Inception, where Inception modules ...

☆ 99 Cited by 606 Related articles All 6 versions »

Xception: Deep Learning with Depthwise Separable Convolutions

François Chollet
Google, Inc.
fchollet@google.com

Abstract

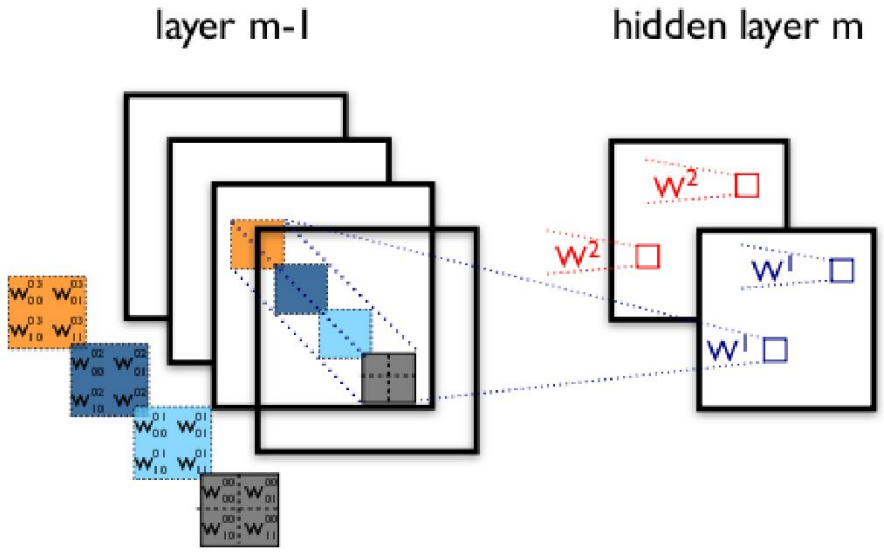
We present an interpretation of Inception modules in convolutional neural networks as being an intermediate step in-between regular convolution and the depthwise separable convolution operation (a depthwise convolution followed by a pointwise convolution). In this light, a depthwise separable convolution can be understood as an Inception module with a maximally large number of towers. This observation leads us to propose a novel deep convolutional neural network architecture inspired by Inception, where Inception modules

as GoogLeNet (Inception V1), later refined as Inception V2 [7], Inception V3 [21], and most recently Inception-ResNet [19]. Inception itself was inspired by the earlier Network-In-Network architecture [11]. Since its first introduction, Inception has been one of the best performing family of models on the ImageNet dataset [14], as well as internal datasets in use at Google, in particular JFT [5].

The fundamental building block of Inception-style models is the Inception module, of which several different versions exist. In figure 1 we show the canonical form of an Inception module, as found in the Inception V3 architec-

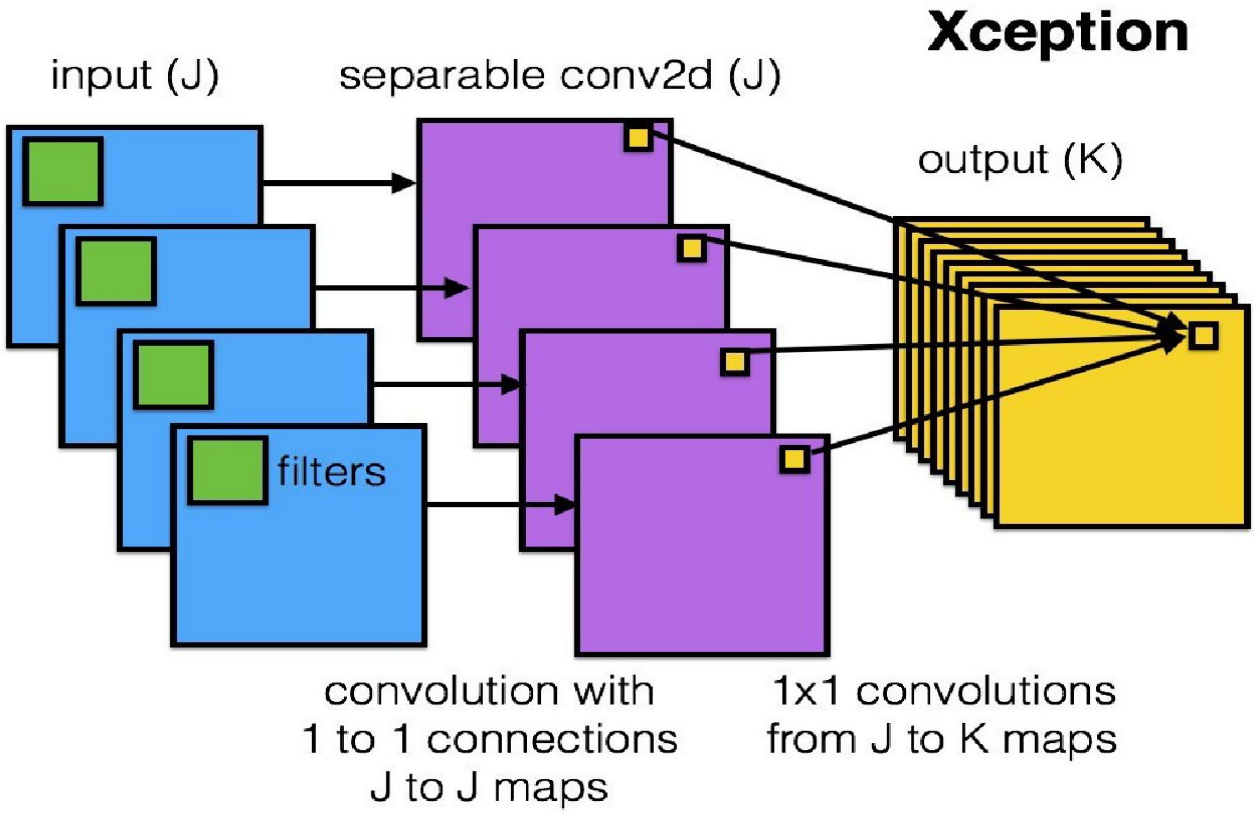
Xception Network

- **Xception by Google**, stands for Extreme version of Inception
- With a modified ***depthwise separable convolution***, it is **even better than Inception-v3**
- In a traditional conv net, convolutional layers seek out correlations across both *space* and *depth*.



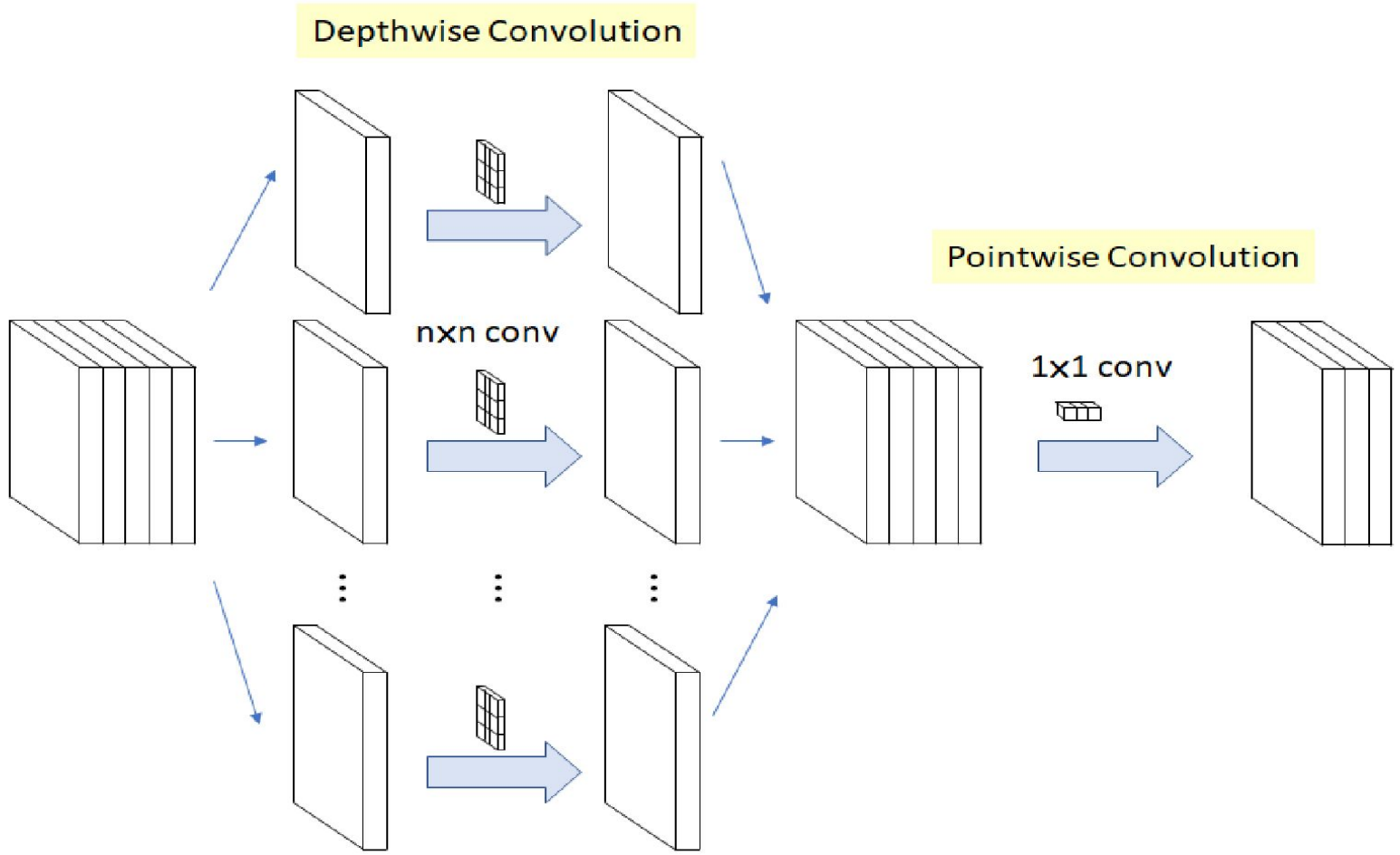
Xception

- In Xception, it maps the spatial correlations for *each output channel separately*, and then performs a 1x1 depthwise convolution to capture cross-channel correlation.



Xception

- Depth-wise Separable Convolutions

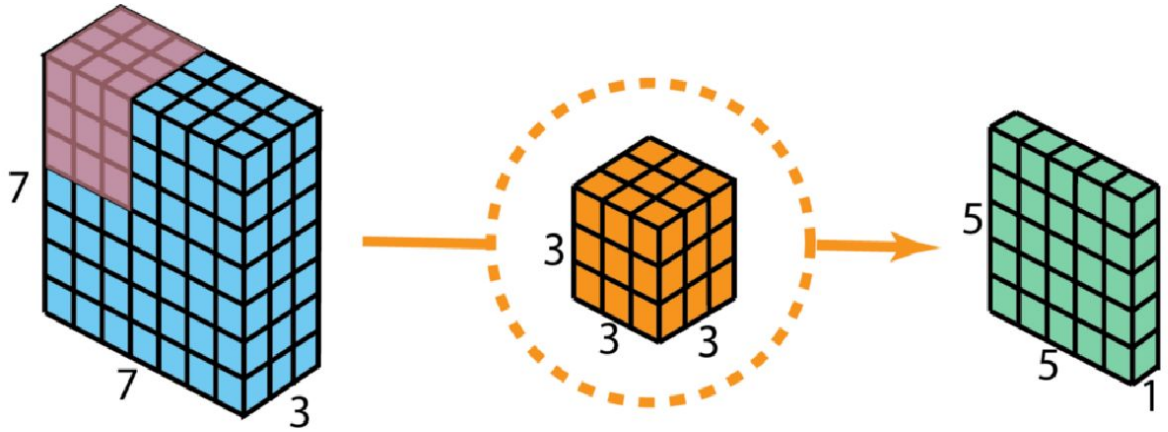


Xception

- Depthwise separable convolution is the **depthwise convolution followed by a pointwise convolution**.
  - **Depthwise convolution** is the **channel-wise n×n spatial convolution**. Suppose we have 5 channels, then we will have 5 n×n spatial convolutions.
  - **Pointwise convolution** actually is the **1×1 convolution** to change the dimension.

Xception: Depth-wise Separable Conv

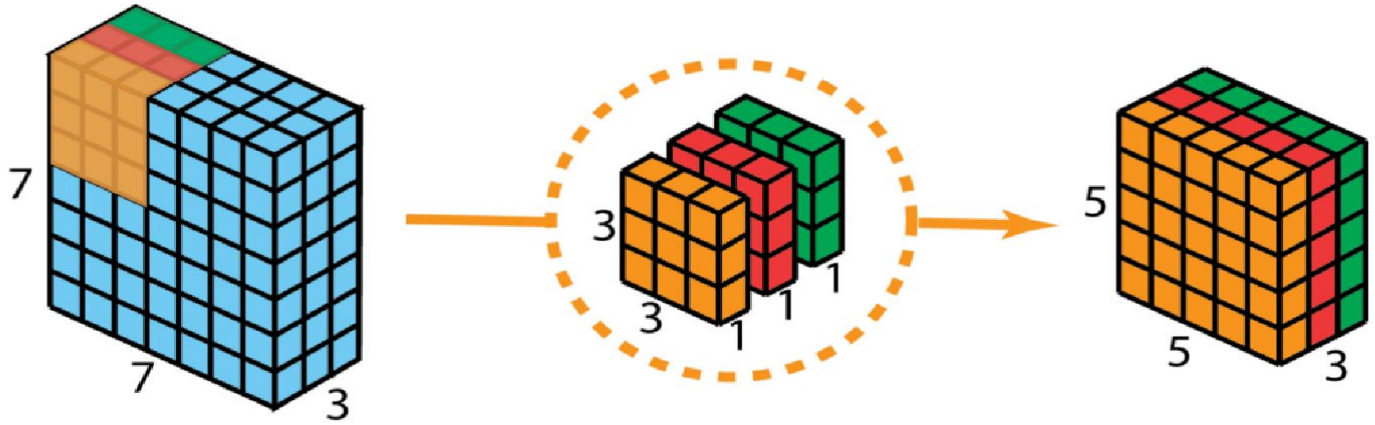
- Standard Convolution



- Let's say the input layer is of size 7 x 7 x 3 (height x width x channels), and the filter is of size 3 x 3 x 3.
- After the 2D convolution with one filter, the output layer is of size 5 x 5 x 1

Xception: Depth-wise Separable Conv

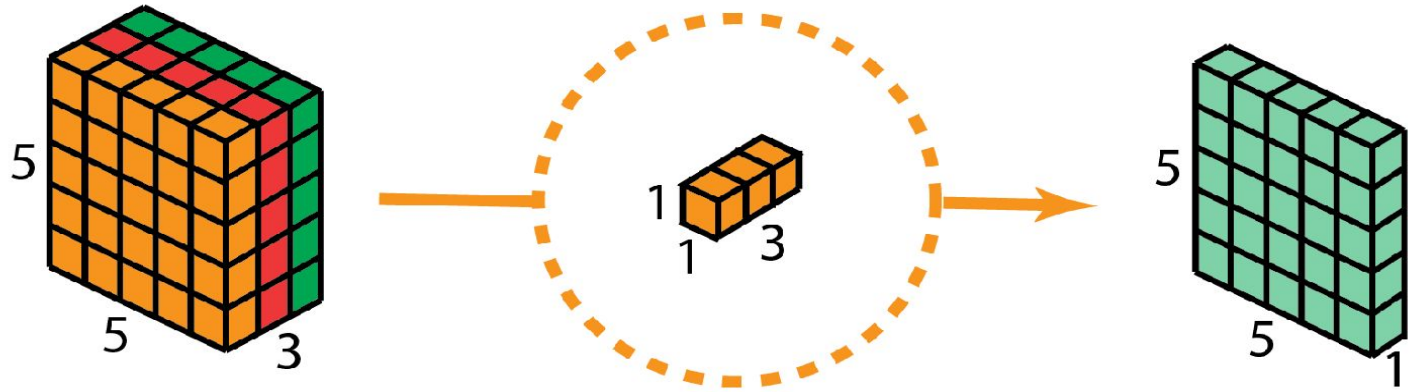
- Depth-wise Separable Convolution



- Instead of using a single filter of size 3 x 3 x 3 in 2D convolution, we used 3 kernels, separately.
- Each filter has size 3 x 3 x 1. Each kernel convolves with 1 channel of the input layer (1 channel only, not all channels!). Each of such convolution provides a map of size 5 x 5 x 1.
- We then stack these maps together to create a 5 x 5 x 3 image. After this, we have the output with size 5 x 5 x 3.

Xception: Depth-wise Separable Conv

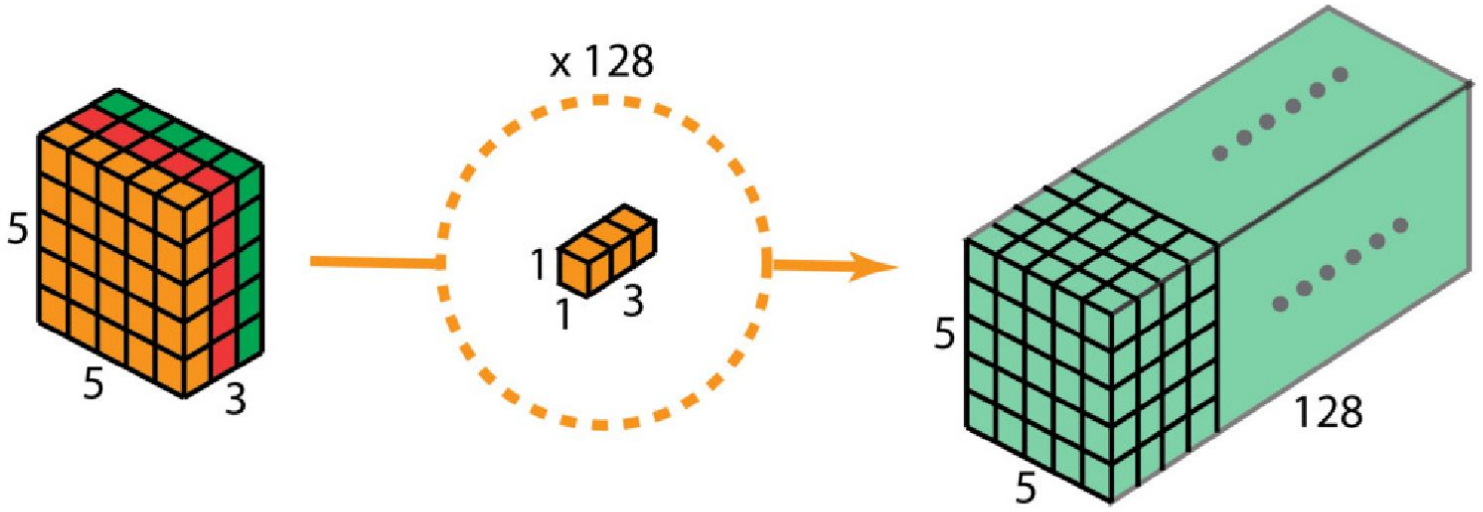
- Depth-wise Separable Convolution



- As the second step of depthwise separable convolution, we apply the 1x1 convolution with kernel size 1x1x3.
- Convolving the 5 x 5 x 3 input image with each 1 x 1 x 3 kernel provides a map of size 5 x 5 x 1.

Xception: Depth-wise Separable Conv

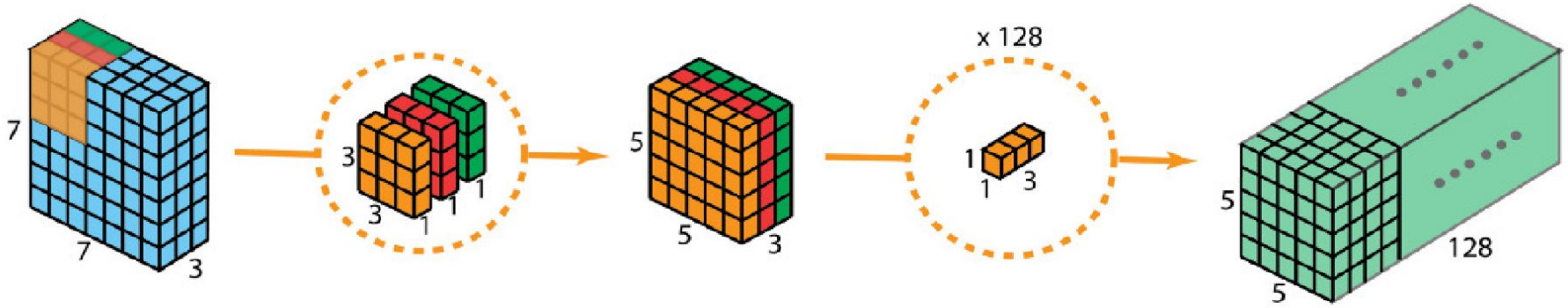
- Depth-wise Separable Convolution



- After applying 128 1x1 convolutions, we can have a layer with size 5 x 5 x 128.

Xception: Depth-wise Separable Conv

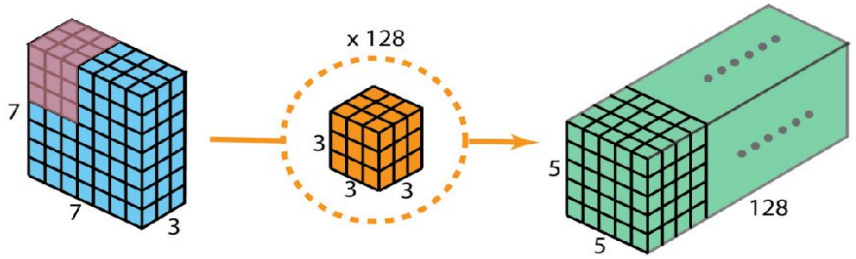
- Depth-wise Separable Convolution



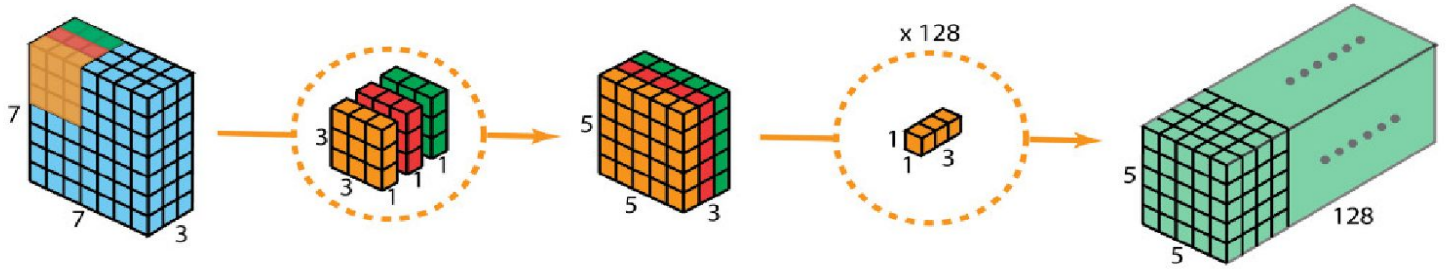
- With these two steps, depth-wise separable convolution also transform the input layer (7 x 7 x 3) into the output layer (5 x 5 x 128).
- Gain?
  - Lesser Computations

Xception: Efficiency

- 2D Conv: There are 128 3x3x3 kernels that move 5x5 times.
- That is 3 x 3 x 3 x 5 x 5 x 128 = 86,400 multiplications.



- In the first depthwise convolution step, there are 3 3x3x1 kernels that moves 5x5 times: 3x3x1x5x5x3 = 675 multiplications.
- In the second step of 1 x 1 convolution, there are 128 1x1x3 kernels that moves 5x5 times: 1 x 1 x 3 x 5 x 5 x 128 = 9,600 multiplications.
- Thus, overall, the depthwise separable convolution takes 675 + 9600 = 10,275 multiplications. This is only about 12% of the cost of the 2D convolution!

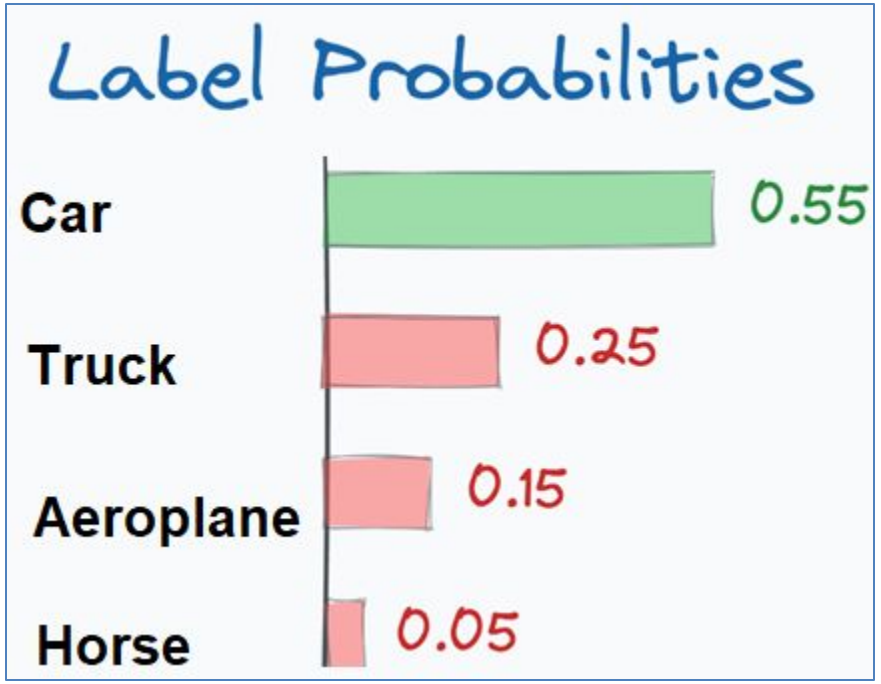


Evaluation Metrics

- Accuracy is a common metric for classification models.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

- **Top-K Categorical Accuracy** (for large class datasets, like ImageNet):
  - Checks if the correct label is in the top K predictions.



Evaluation Metrics

- Precision and Recall (for imbalanced data):

> Precision measures how many of the predicted "positive" cases were actually correct.

📌 **Formula:**

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **True Positives (TP):** Cases correctly predicted as positive.
- **False Positives (FP):** Cases incorrectly predicted as positive.

🔹 **High Precision → Fewer False Positives**

Evaluation Metrics

Recall measures how many of the actual positive cases were correctly identified.

- **Formula:**

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

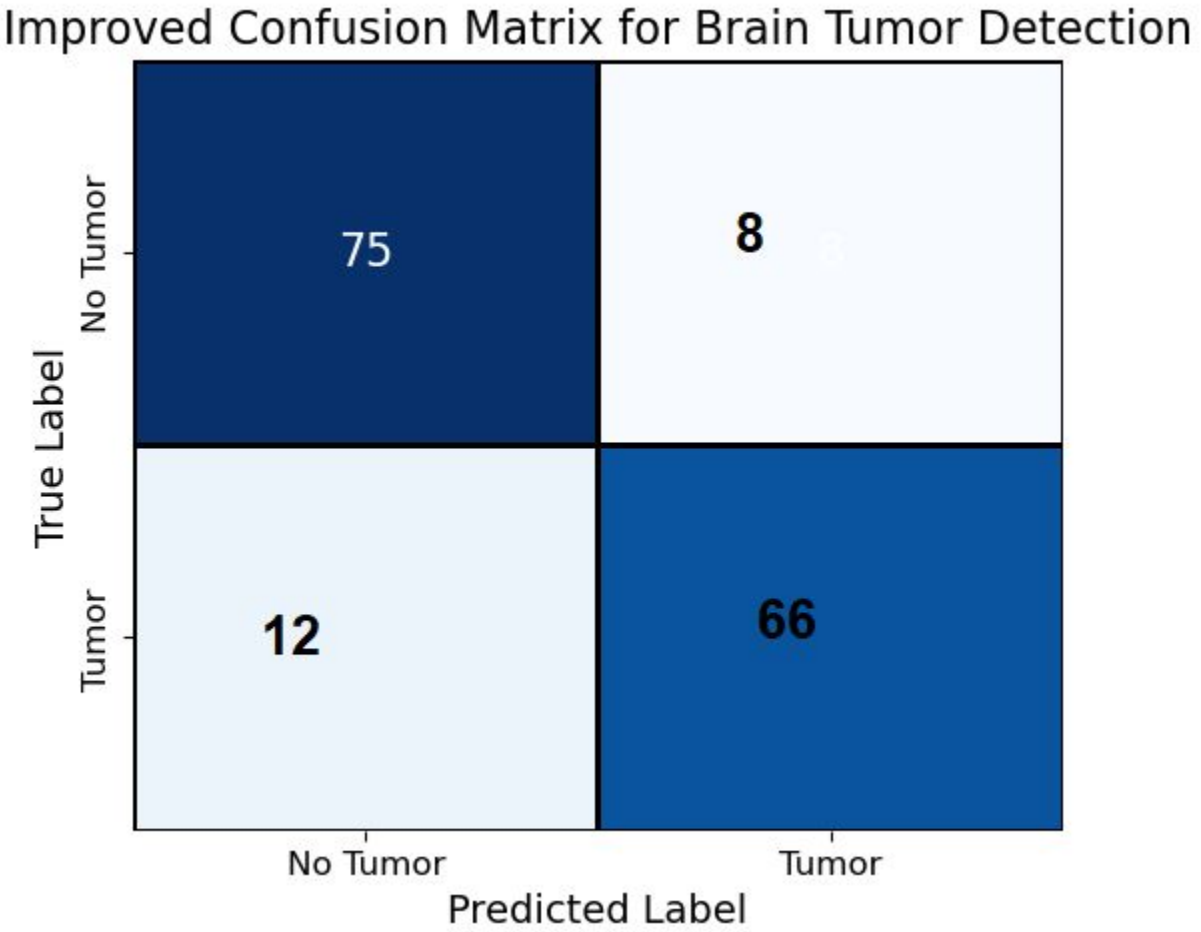
- **False Negatives (FN):** Cases that should have been positive but were misclassified as negative.
- **High Recall → Fewer False Negatives**
- If Recall is **high**, the model is good at **finding all positive cases**.

📌 **Example - Medical Diagnosis (Cancer Detection)**

Scenario	Precision	Recall
Doctors diagnosing cancer	⚠️ Low Precision (many non-cancer cases falsely diagnosed)	✅ High Recall (almost all real cancer cases detected)
Spam detection	✅ High Precision (only real spam is marked)	⚠️ Low Recall (some spam emails go undetected)

Evaluation Metrics

- **Confusion Matrix**
- Evaluate model performance by comparing the actual labels with the predicted labels.
- How well the model is distinguishing between different classes.



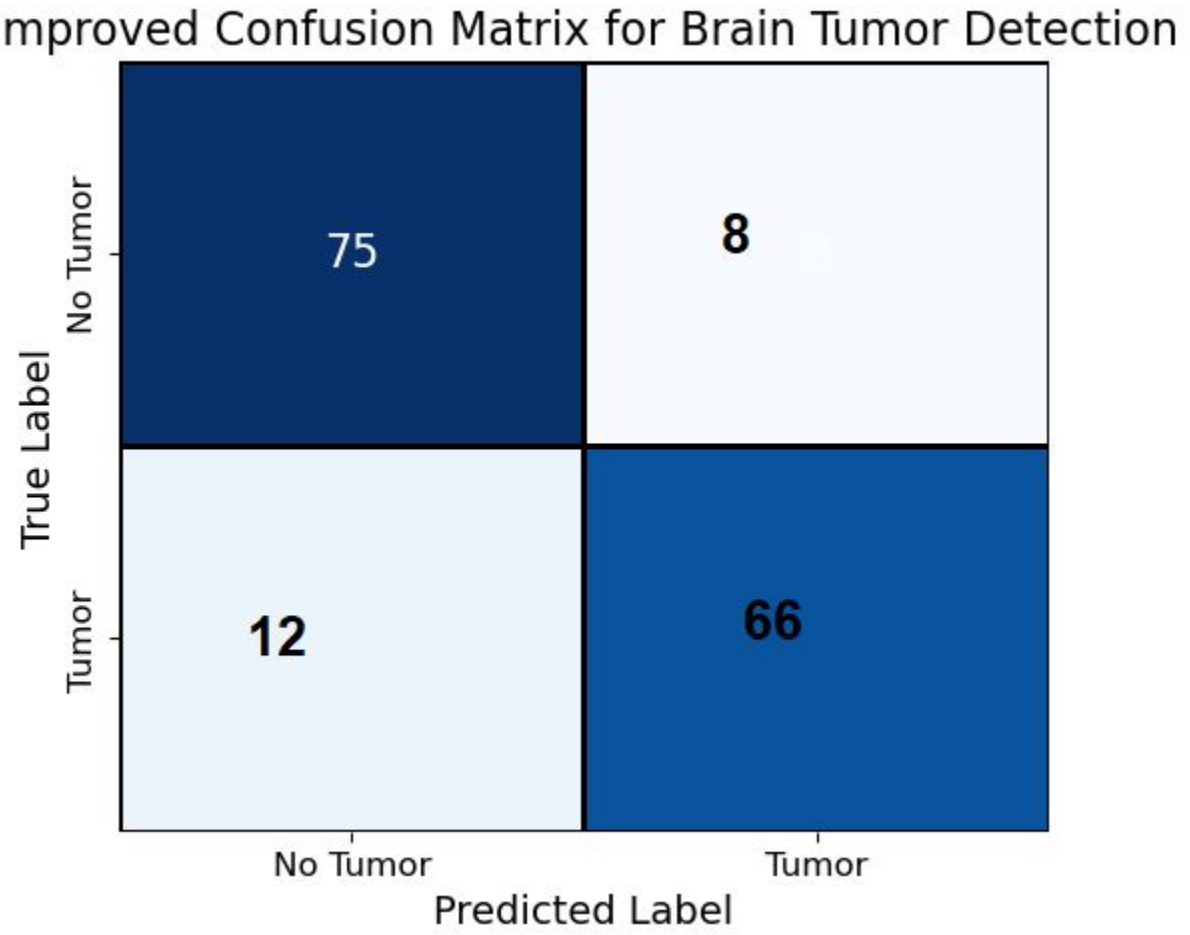
Evaluation Metrics

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Precision} = \frac{66}{66 + 8} = \frac{66}{74} \approx 0.8919\,(89.19\%)$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Recall} = \frac{66}{66 + 12} = \frac{66}{78} \approx 0.8462\,(84.62\%)$$



Thank You ☺